Dan CALAMIA
Septembre 2026

Master 2 – Systèmes Avances et Robotique (SAR)
Cursus Master en Ingénierie (CMI)

Rapport de stage de fin d'études

Cartographie sémantique coopérative multi-robots par fusion LiDAR-caméra

Extension sémantique de LIO-SAM pour la coopération inter-robots dans le cadre du projet ANR-SOS

Laboratoire d'accueil :
CRIStAL

Période de stage :
du 27/04 au 27/09 2026

Encadrant académique :
Faïz Ben AMAR

Encadrant laboratoire :
Cindy CAPPELLE

Sorbonne Université
Année universitaire Septembre 2025-2026

Remerciements

Je tiens tout d'abord à remercier Mme Cindy Cappelle, Maîtresse de conférences et mon encadrante au laboratoire CRIStAL, pour son encadrement tout au long de ce stage, sa disponibilité et ses conseils, qui m'ont permis de progresser aussi bien sur les aspects techniques que sur la structuration de ce travail.

Mes remerciements vont aussi à l'ensemble de l'équipe du laboratoire CRIStAL pour son accueil et la bonne ambiance de travail qui a accompagné ce stage. Je remercie également Clovis Adeler doctorant au laboratoire, pour son aide au quotidien tout au long de ce stage et Gérald Dherbomez, ingénieur de recherche et responsable technique de la plateforme PRETIL, pour son aide précieuse lors des acquisitions réalisées avec le robot.

Enfin, je remercie mes proches pour leur soutien tout au long de ce stage.

Table des matières

1 Introduction **5**
1.1 Contexte du stage . . . 5
1.1.1 Contexte du laboratoire d'acceuil . . . 5
1.1.2 Sujet du stage . . . 5

2 Cardre théorique **6**
2.1 SLAM . . . 6
2.1.1 Full SLAM . . . 7
2.1.2 Online SLAM . . . 7
2.2 Résolution du problème du FULL SLAM . . . 7
2.2.1 Graphe de facteurs . . . 7

3 État de l'art **9**
3.1 Panorama des systèmes de SLAM . . . 9
3.1.1 ORB-SLAM3 . . . 9
3.1.2 VINS-Fusion . . . 10
3.1.3 Kimera . . . 10
3.1.4 KISS-ICP . . . 10
3.1.5 LOAM . . . 10
3.1.6 Point-LIO . . . 10
3.1.7 FAST-LIO2 . . . 11
3.1.8 Faster-LIO . . . 11
3.1.9 DLIO . . . 11
3.1.10 LeGO-LOAM . . . 11
3.1.11 SuMa++ . . . 12
3.1.12 Cartographer . . . 12
3.1.13 RTAB-Map . . . 12
3.1.14 LIO-SAM . . . 13
3.2 LIOSAM . . . 13
3.3 YOLO . . . 15

4 Framework pour la cartographie sémantique par fusion LiDAR-caméra **17**
4.1 Architecture générale . . . 18
4.2 Acquisition et projection des données sur l'image . . . 18
4.2.1 Synchronisation temporelle des flux . . . 18
4.2.2 Projection géométrique des points LiDAR sur l'image . . . 18
4.2.3 Segmentation sémantique via YOLO et association aux points . . . 19
4.3 Passage dans le repère monde . . . 20
4.3.1 Synchronisation temporelle . . . 20
4.3.2 Queue de fonctionnement . . . 20

4.3.3 Interpolation de l'odométrie 20
4.4 Parseur et générateur de carte 20
4.5 Tests et vérifications 21
4.5.1 Installation et configuration des données KITTI 21
4.5.2 Validation de LIO sur les données KITTI 23
4.5.3 Validation du framework complet 24

5 Loop closure **26**
5.1 Détection de loop closure depuis LIO 26
5.1.1 Paramètres modifiables dans LIO 26
5.2 Évolution de l'architecture 27
5.2.1 Détection d'une loop closure 28
5.2.2 Mise à jour des points 28
5.3 Tests et validations 29
5.3.1 Test de la fermeture de boucle sur les données KITTI 29

6 Carte à-priori **30**
6.1 Prélèvement de la carte 30
6.2 Discrétisation de la carte a priori 30
6.2.1 Classification sémantique des cellules 31
6.3 Format et exportation 31
6.4 Implementation de la carte dans l'algorithme 32

7 Extension multi-robots **34**
7.1 Nouvelle architecture 34
7.1.1 Séparation en deux nœuds 34
7.1.2 Gestion des topics et des repères TF entre robots 34
7.1.3 Publication et agrégation des points étiquetés 35
7.1.4 Génération de carte périodique 35

8 Expérimentation et résultats **36**
8.1 Expériences 36
8.1.1 Matériels expérimentaux 36
8.1.2 Configuration des acquisitions 37
8.2 Résultats obtenus 37
8.2.1 Ranger 37
8.2.2 Scout 38
8.2.3 Multi-robots 39

9 Suites et améliorations **41**
9.1 Deux modèles modulaires 41
9.2 Position et orientation initiale 41
9.3 Fusion Bayésienne 42
9.4 Tests en conditions réelles 42
9.4.1 Entraînement de YOLO 42

10 Conclusion **43**

11 Annexe **44**

Résumé

Ce stage, réalisé au laboratoire CRIStAL dans le cadre du projet ANR-SOS, porte sur la cartographie sémantique coopérative multi-robots par fusion LiDAR-caméra. Un framework a été développé permettant de projeter les nuages de points LiDAR sur les images caméra, de les étiqueter sémantiquement à l'aide de YOLO. Les cartes sémantiques obtenues, exportées au format GeoJSON, peuvent être enrichies à l'aide d'une carte a priori et étendues à des scénarios multi-robots. L'approche a été validée sur le jeu de données KITTI ainsi que sur des acquisitions terrain réalisées avec deux robots terrestres AgileX (Scout Mini et Ranger), démontrant son applicabilité en conditions réelles, y compris la prise en compte des fermetures de boucle.

Abstract

This internship, carried out at the CRIStAL laboratory as part of the ANR-SOS project, focuses on cooperative semantic mapping for multi-robot systems through LiDAR-camera fusion. A framework was developed that projects LiDAR point clouds onto camera images and labels them semantically using YOLO. The resulting semantic maps, exported as GeoJSON grids, can be enriched with prior map information and extended to multi-robot scenarios. The approach was validated on the KITTI dataset as well as on field acquisitions with two AgileX ground robots (Scout Mini and Ranger), demonstrating its applicability to real-world conditions, including loop-closure handling.

Chapitre 1

Introduction

1.1 Contexte du stage

La cartographie est une brique essentielle de la robotique : elle permet à un robot de se localiser et de représenter l'environnement dans lequel il évolue. Dans certains cas, cette cartographie ne peut plus se limiter à une représentation géométrique de l'espace libre et occupé. Elle doit intégrer une information *sémantique* (nature des objets, des zones, des obstacles rencontrés) afin d'être exploitable à la fois pour la navigation autonome et pour la prise de décision de haut niveau. Celle ci doit également pouvoir être partagée et fusionnée entre plusieurs robots dont les conditions de communication sont imparfaites.

1.1.1 Contexte du laboratoire d'acceuil

Ce stage a été réalisé au sein du laboratoire CRIStAL (Centre de Recherche en Informatique, Signal et Automatique de Lille), unité mixte de recherche (UMR 9189) placée sous la tutelle du CNRS, de l'Université de Lille, de Centrale Lille et d'Inria. Le stage s'est déroulé au sein de l'équipe ToSyMA (*Tolérance aux fautes des Systèmes Mobiles Autonomes*) dont les travaux portent notamment sur le diagnostic et l'observation d'état, la fusion multi-capteurs tolérante aux fautes pour la localisation, ainsi que la sûreté de fonctionnement de systèmes dynamiques distribués et coopératifs. Il s'est déroulé sous l'encadrement de Mme Cindy Cappelle, Maîtresse de conférences.

1.1.2 Sujet du stage

Ce stage s'inscrit dans le projet ANR-SOS, dont l'objectif est de développer des capacités de cartographie et de coopération pour des flottes de robots mobiles évoluant en environnement extérieur, potentiellement sans connectivité continue entre les agents. Mon travail, réalisé au sein du laboratoire CRIStAL, porte spécifiquement sur la brique de cartographie sémantique : la construction d'une carte enrichie sémantiquement à partir de la fusion des données LiDAR et caméra d'un robot, puis l'extension de cette brique à un contexte multi-robots.

Ce rapport présente dans un premier temps le cadre théorique des outils utilisés et l'état de l'art des systèmes de SLAM sur lesquels s'appuient ces travaux, avant de détailler le travail réalisé : mise en place de la fusion sémantique caméra-LiDAR, construction d'une carte a priori, prise en compte des fermetures de boucle et extension au multi-robots, puis les résultats obtenus.

Chapitre 2

Cardre théorique

2.1 SLAM

Simultaneous Localization and Mapping (SLAM), ou Localisation et Cartographie Simultanées, est un processus complexe car pour pouvoir se repérer dans un environnement on a besoin d'une carte, et pour pouvoir créer une carte on a besoin de savoir où l'on se trouve. Le problème du SLAM va donc essayer de résoudre ces deux inconnues simultanément, à partir des mêmes données capteurs.

Notons $X_T = \{x_0, x_1, \ldots, x_T\}$ les positions du robot (sa trajectoire), $U_T = \{u_0, u_1, \ldots, u_T\}$ les mesures d'odométrie entre les instants $t-1$ et t, $Z_T = \{z_0, z_1, \ldots, z_T\}$ les mesures reliant des amers* observés à la carte m et à la pose x_t du robot, et m la carte réelle de l'environnement, à déterminer.

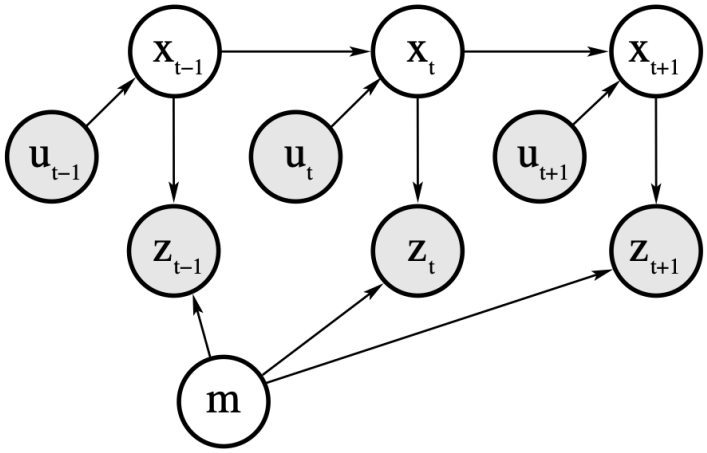


FIGURE 2.1 – Modèle graphique du problème de SLAM. Les arcs indiquent des relations de causalité, et les nœuds grisés sont directement observables par le robot. Dans le SLAM, le robot cherche à retrouver les variables non observables.

Deux types de formulations du problème SLAM existent, selon que l'on cherche à estimer l'ensemble de la trajectoire passée ou seulement la position courante du robot.

∗. un amer (landmark en anglais) est un élément fixe et identifiable de l'environnement, un point, un coin, un objet repérable, que le robot observe à plusieurs reprises au cours de ses déplacements, et dont la position, estimée conjointement avec la trajectoire du robot, sert de repère commun pour corriger la dérive de l'odométrie

2.1.1 Full SLAM

Le *Full SLAM* consiste à estimer la probabilité a posteriori sur l'ensemble de la trajectoire du robot ainsi que sur la carte :

$$p(X_T, m \mid Z_T, U_T)$$

Cette formulation est résolue par une optimisation globale (*pose graph optimization*), qui ajuste conjointement l'ensemble des poses passées et la carte. Elle rend possible la prise en compte des fermetures de boucle (*loop closure*) : lorsqu'un lieu déjà visité est reconnu, une contrainte supplémentaire est ajoutée au graphe, ce qui permet de corriger rétroactivement toute la trajectoire passée. Le Full SLAM est donc précis, mais plus coûteux en calcul, le graphe grandissant avec la durée de la mission.

2.1.2 Online SLAM

L'*Online SLAM* (par filtres) consiste à n'estimer que la position actuelle du robot ainsi que la carte :

$$p(x_t, m \mid Z_t, U_t)$$

Cette formulation est résolue par un filtrage séquentiel récursif, dérivé du filtre de Bayes, qui met à jour l'estimation courante à chaque nouvelle mesure sans revenir sur les poses passées. Plusieurs familles de filtres sont utilisées selon les hypothèses faites sur le bruit et la linéarité du système : le filtre de Kalman (EKF/UKF pour Extended/Unscented Kalman Filter) dans le cas de bruits gaussiens et de modèles peu ou modérément non linéaires, ou le filtre particulaire (*particle filter*), qui représente la distribution de probabilité par un ensemble de particules plutôt que par une loi gaussienne, ce qui le rend adapté à des distributions multimodales ou fortement non linéaires, au prix d'un coût de calcul plus élevé lorsque le nombre de particules augmente. Ces approches sont relativement légère et compatibles avec un fonctionnement temps réel, mais au prix d'une dérive cumulative (*drift*) : les erreurs d'estimation s'accumulent au fil du temps sans correction rétroactive possible.

2.2 Résolution du problème du FULL SLAM

Pour la compréhension des choix techniques de ce stage, détaillés dans le chapitre suivant, il est nécessaire de comprendre le fonctionnement des graphes de facteurs, qui permettent de représenter graphiquement le problème mathématique du Full SLAM.

En pratique, cependant, ces deux approches ne s'opposent pas totalement : les algorithmes reposant sur un solveur de graphe s'appuient souvent également, en amont, sur un filtre, typiquement pour fournir une estimée initiale de la trajectoire entre deux optimisations globales. Les deux méthodes se combinant plutôt qu'elles ne s'excluent.

2.2.1 Graphe de facteurs

Un graphe de facteurs est un modèle graphique composé de deux types de nœuds : des nœuds de variables, qui représentent les inconnues à estimer (ici, les poses successives $x_0, x_1, \ldots, x_T$ du robot), et des nœuds de facteurs, qui représentent chacun une contrainte probabiliste issue d'une mesure capteur et relient entre elles les variables qu'elle concerne. Résoudre le Full SLAM revient alors à chercher la configuration de poses qui maximise

la probabilité a posteriori $p(X_T, m \mid Z_T, U_T)$ compte tenu de l'ensemble des facteurs, ce qui équivaut, sous hypothèse de bruits gaussiens, à minimiser la somme des erreurs quadratiques entre chaque mesure et sa prédiction par le modèle correspondant.Par exemple, GTSAM ou g2o implémentent une optimisation efficace et creuse des graphes de facteurs pour le SLAM en temps réel.

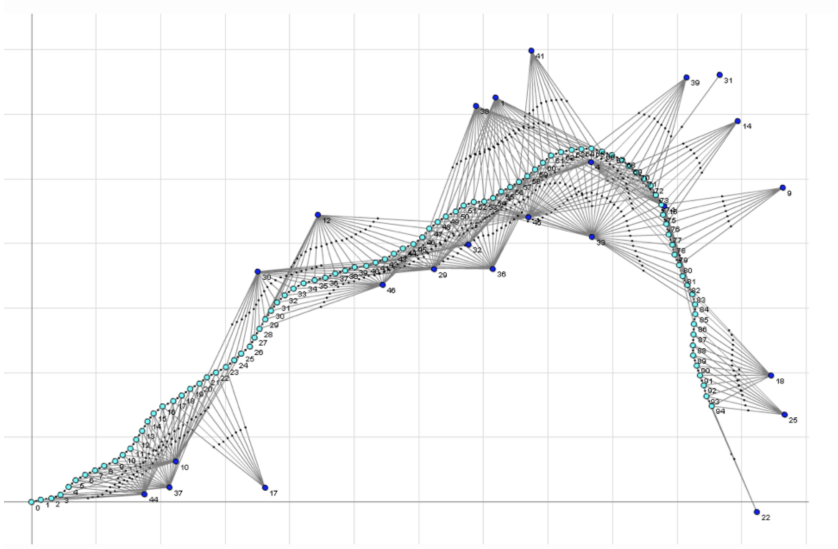


FIGURE 2.2 – Exemple de graphe de facteurs ([Dellaert, 2005])

La figure 2.2 illustre un exemple de graphe de facteurs : les nœuds en bleu clair représentent les poses clés du robot, reliées entre elles par les facteurs (contraintes) d'odométrie successives qui dessinent la trajectoire. Les nœuds en bleu foncé quant à eux représentent les amers, ils sont reliés aux poses du robot par des facteurs (contraintes) liés aux mesures LiDAR. L'optimisation globale ajuste l'ensemble de ces facteurs simultanément pour minimiser l'incohérence globale du graphe.

Chapitre 3

État de l'art

Ce chapitre présente l'état de l'art des briques technologiques sur lesquelles s'appuie ce travail. Il commence par un panorama comparatif des principaux systèmes de SLAM, afin de justifier le choix retenu pour ce stage, puis détaille son fonctionnement. Ce système ne fournissant qu'une carte géométrique, dépourvue d' information sémantique, une seconde brique est nécessaire pour venir l'enrichir : le modèle de détection YOLO, présenté en fin de chapitre.

3.1 Panorama des systèmes de SLAM

Système	**Entrées**	**Sortie**	**Optimisation**	**Loop closure**
ORB-SLAM3	Caméra (+ IMU)	Trajectoire + carte éparse	Bundle adjustment	Oui (BoW)
VINS-Fusion	Caméra + IMU	Trajectoire + carte éparse	Sliding window BA	Oui
Kimera	Caméra stéréo + IMU	Trajectoire + mesh + sémantique	Graphe robuste (RPGO)	Oui
KISS-ICP	LiDAR seul	Odométrie	ICP robuste	Non
LOAM	LiDAR	Odométrie + carte	Moindres carrés	Non
Point-LIO	LiDAR + IMU	Odométrie (point-à-point) + carte	iEKF point-à-point	Non (natif)
Faster-LIO	LiDAR + IMU	Odométrie + carte	iEKF (iVox)	Non (natif)
DLIO	LiDAR + IMU	Odométrie + carte	GICP + déskew continu	Non (natif)
FAST-LIO2	LiDAR + IMU	Odométrie + carte	iEKF (ikd-Tree)	Non (natif)
LeGO-LOAM	LiDAR (+ IMU)	Odométrie + carte	Moindres carrés	ICP simple
SuMa++	LiDAR + segmentation sémantique	Trajectoire + carte surfel sémantique	ICP projectif (surfels)	Oui
Cartographer	LiDAR (+ IMU, roues)	Carte + trajectoire	Scan matching + graphe	Oui
RTAB-Map	RGB-D/Stéréo/LiDAR	Nuage/mesh + grille occ.	Graphe (g2o/GTSAM)	Oui (BoW visuel)
LIO-SAM	LiDAR + IMU 9 axes (+ GPS)	Odométrie + carte	Graphe de facteurs (GTSAM)	Oui
LIORF	LiDAR + IMU 6 axes (+ GPS)	Odométrie + carte	Graphe de facteurs (GTSAM)	Oui

TABLE 3.1 – Panorama comparatif des principaux systèmes de SLAM identifiés, du moins adapté au plus adapté aux besoins de ce stage.

3.1.1 ORB-SLAM3

ORB-SLAM3 (Campos et al. [2021]) fonctionne à partir d'une caméra monoculaire, stéréo ou RGB-D, avec une IMU optionnelle, et produit la trajectoire de la caméra ainsi qu'une carte éparse de points 3D (amers ORB). Il repose sur un SLAM visuel, éventuellement inertiel, fondé sur des caractéristiques ORB, avec une gestion multi-cartes (*Atlas*) permettant de relocaliser le robot après une perte de suivi ; l'ensemble est optimisé par ajustement de faisceaux (*bundle adjustment*), la fermeture de boucle étant assurée par un sac de mots visuels (DBoW2). Il est utilisé sur des robots ou des drones équipés uniquement de caméra(s), sans LiDAR.

3.1.2 VINS-Fusion

VINS-Fusion (Qin et al. [2018]) prend en entrée une caméra mono ou stéréo couplée à une IMU, avec un GPS optionnel, et produit une odométrie visuelle-inertielle ainsi qu'une carte éparse de points. Il s'initialise par un alignement visuel-inertiel, puis optimise la trajectoire par fenêtre glissante (*sliding window bundle adjustment*), la fermeture de boucle étant obtenue par relocalisation. Il est très utilisé en robotique aérienne, sur des drones et des robots légers ne disposant pas de LiDAR.

3.1.3 Kimera

Kimera (Rosinol et al. [2020]) fonctionne à partir d'une caméra stéréo couplée à une IMU, et produit à la fois une trajectoire, un mesh 3D métrique et une carte sémantique, avec une étiquette associée à chaque face du mesh ou à chaque voxel. Il s'organise en quatre modules : *Kimera-VIO* pour l'odométrie visuelle-inertielle, *Kimera-RPGO* pour l'optimisation robuste du graphe de pose, *Kimera-Mesher* pour la reconstruction du mesh 3D, et *Kimera-Semantics* pour la fusion sémantique, qui propage les étiquettes 2D vers une structure voxel en 3D.

3.1.4 KISS-ICP

KISS-ICP (Vizzo et al. [2023]) ne prend en entrée qu'un LiDAR, sans IMU, et produit uniquement une odométrie LiDAR. Il repose sur un recalage ICP point-à-point robuste, avec des seuils adaptatifs et sans extraction de caractéristiques complexe, et a été conçu pour ne nécessiter quasiment aucun réglage de paramètres. Il sert avant tout de référence de comparaison (*baseline*) simple et robuste, notamment lorsque l'IMU est indisponible ou peu fiable.

3.1.5 LOAM

LOAM (Zhang and Singh [2014]) est le système historique de référence du domaine. Il ne prend en entrée qu'un nuage de points LiDAR 3D, sans IMU dans sa version originale, et produit en sortie une odométrie (pose à 6 degrés de liberté) à haute fréquence ainsi qu'une carte de points accumulée. Son fonctionnement repose sur l'extraction de points caractéristiques – des arêtes et des plans – identifiés par l'analyse de la courbure locale du nuage, suivie d'un recalage successif scan-to-scan puis scan-to-map, le tout optimisé par moindres carrés ; il ne dispose en revanche d'aucun mécanisme de fermeture de boucle natif. LOAM sert aujourd'hui essentiellement de base méthodologique à de nombreux systèmes dérivés, tels que LeGO-LOAM, LIO-SAM ou A-LOAM.

3.1.6 Point-LIO

Point-LIO (He et al. [2022]) prend en entrée un LiDAR 3D et une IMU, mais se distingue par un traitement point par point plutôt que par scan complet : chaque point LiDAR et chaque mesure IMU sont intégrés individuellement dans le filtre au fur et à mesure de leur arrivée, ce qui produit une estimation de pose à très haute fréquence (fréquence des points eux-mêmes) et évite les artefacts de distorsion liés au dé-biaisage par scan. Cette approche le rend particulièrement robuste aux mouvements agressifs et aux vitesses angulaires élevées, typiquement sur des plateformes très dynamiques ou des

LiDAR à rotation rapide – un profil d'usage différent de celui de ce stage – et il ne propose pas de fermeture de boucle native.

3.1.7 FAST-LIO2

FAST-LIO2 (Xu et al. [2022]) prend en entrée un LiDAR 3D – y compris les capteurs solid-state de type Livox – couplé à une IMU, et produit une odométrie à très haute fréquence ainsi qu'une carte. Il s'appuie sur un filtre de Kalman étendu itéré (iEKF) fortement couplé, et sur une structure de données incrémentale appelée *ikd-Tree* permettant une recherche de voisins particulièrement rapide ; il ne dispose pas nativement de fermeture de boucle, celle-ci n'existant que sous forme d'extension. Il est privilégié sur les plateformes exigeant une faible latence, comme les drones, et sur les capteurs LiDAR solid-state, et constitue l'alternative directe à LIO-SAM la plus souvent citée en comparaison – l'absence de fermeture de boucle native restant toutefois un manque important pour ce stage.

3.1.8 Faster-LIO

Faster-LIO (Bai et al. [2022]) prend en entrée un LiDAR 3D et une IMU, et produit une odométrie ainsi qu'une carte, à la manière de FAST-LIO2 dont il est directement issu. Il repose sur le même filtre de Kalman étendu itéré (iEKF) fortement couplé, mais remplace la structure incrémentale *ikd-Tree* par une structure de voxels incrémentale appelée *iVox*, ce qui accélère encore la recherche de voisins et réduit la charge de calcul. Il ne dispose pas non plus de fermeture de boucle native. Il est particulièrement adapté aux plateformes embarquées disposant de ressources de calcul très limitées, là où FAST-LIO2 reste encore trop coûteux.

3.1.9 DLIO

DLIO (Chen et al. [2023]) prend en entrée un LiDAR 3D et une IMU, et produit une odométrie ainsi qu'une carte. Il se distingue par un dé-biaisage continu du nuage de points, calculé analytiquement à partir du modèle de mouvement de l'IMU plutôt que par une correction discrète classique, couplé à un recalage rapide par GICP (*Generalized ICP*) et à une structure de carte optimisée pour la recherche de voisins. Il ne dispose pas non plus de fermeture de boucle native. Il constitue, avec FAST-LIO2, l'une des références récentes les plus citées en matière de précision et d'efficacité de calcul pour l'odométrie LiDAR-inertielle.

3.1.10 LeGO-LOAM

LeGO-LOAM (Shan and Englot [2018]) prend en entrée un LiDAR 3D de type Velodyne, associé à une IMU optionnelle utilisée pour le dé-biaisage du nuage de points (*deskewing*), et produit une odométrie et une carte optimisées pour un robot terrestre. Il fonctionne en segmentant d'abord le sol des obstacles avant d'extraire les points caractéristiques, ce qui allège fortement le calcul par rapport à LOAM, puis en optimisant la transformation en deux étapes – d'abord par rapport au sol, puis de façon complète – avec une fermeture de boucle simple réalisée par ICP. Il est particulièrement adapté aux robots terrestres disposant de ressources de calcul limitées, et constitue le prédécesseur direct de LIO-SAM.

3.1.11 SuMa++

SuMa++ (Chen et al. [2019]) est particulièrement intéressant pour ce stage puisqu'il s'agit, contrairement à Kimera, d'un système de SLAM sémantique directement construit autour d'un LiDAR plutôt que d'une caméra stéréo. Il prend en entrée un nuage de points LiDAR ainsi que sa segmentation sémantique par point (typiquement produite par un réseau externe comme RangeNet++), et produit à la fois la trajectoire du robot et une carte de surfels (*surfel map*) étiquetée sémantiquement. Il étend le système SuMa (*Surfel-based Mapping*) en utilisant l'information sémantique de deux façons : pour filtrer les surfels correspondant à des objets dynamiques (véhicules, piétons) avant le recalage, et pour contraindre la mise en correspondance lors du recalage projectif entre le nuage courant et la carte de surfels, n'appariant que des éléments de même classe sémantique – ce qui améliore à la fois la robustesse du SLAM et la cohérence de la carte produite. SuMa++ a essentiellement été validé sur des scénarios de conduite autonome en extérieur (jeux de données KITTI et SemanticKITTI). Il reste néanmoins un système principalement académique, dépendant d'un réseau de segmentation externe entraîné séparément, sans portage ROS 2 ni support natif du multi-robot, ce qui en fait une source d'inspiration méthodologique plutôt qu'une brique directement réutilisable sur ROS2.

3.1.12 Cartographer

Cartographer (Hess et al. [2016]) accepte en entrée un LiDAR 2D ou 3D, avec IMU et odométrie roues optionnelles, et produit une carte d'occupation en 2D (ou des sous-cartes en 3D) ainsi qu'une trajectoire optimisée. Son fonctionnement repose sur la construction de sous-cartes locales par scan matching, puis sur la détection de fermetures de boucle par recherche de correspondance entre sous-cartes (*branch-and-bound*), avant une optimisation globale du graphe de poses. Il est très répandu pour le SLAM 2D en intérieur – entrepôts, bâtiments – mais existe également en 3D, et est intégré nativement à ROS et ROS 2 ; il reste toutefois davantage orienté vers la cartographie 2D d'intérieur que vers l'odométrie LiDAR-inertielle 3D visée dans ce stage.

3.1.13 RTAB-Map

RTAB-Map (Labbé and Michaud [2019]) accepte en entrée une caméra RGB-D ou stéréo, un LiDAR 2D ou 3D, ainsi qu'une odométrie externe, et produit un nuage de points ou un mesh, une grille d'occupation 2D et un graphe de poses. Il se distingue par une gestion explicite d'une mémoire de travail (*Working Memory*) et d'une mémoire à long terme (*Long-Term Memory*), destinée à maîtriser le coût de calcul sur de grandes surfaces explorées pendant une longue durée ; la fermeture de boucle est assurée par un sac de mots visuels, avec un back-end g2o ou GTSAM. Il est particulièrement adapté à la cartographie intérieure à grande échelle et de longue durée, grâce à sa forte polyvalence multi-capteurs, mais sa représentation « sémantique » reste limitée à la coloration RGB des voxels et reste coûteuse en calcul : la recherche de fermeture de boucle par sac de mots visuels sur l'ensemble de la carte et la reconstruction dense (nuage de points ou mesh) restent plus lourdes qu'une simple odométrie LiDAR-inertielle comme LIO-SAM, et c'est précisément pour limiter ce coût que le mécanisme de mémoire de travail/mémoire à long terme (WM/LTM) a été introduit.

3.1.14 LIO-SAM

LIO-SAM (Shan et al. [2020]) prend en entrée un LiDAR 3D et une IMU (IMU 9 axes), avec un GPS optionnel, et produit une odométrie haute fréquence, des poses clés (*keyframes*) et une carte de points globale optimisée. Son fonctionnement repose sur un couplage serré entre l'IMU et le LiDAR au sein d'un graphe de facteurs (GTSAM), alimenté par quatre types de contraintes : la pré-intégration IMU, l'odométrie LiDAR issue du scan matching, le GPS lorsqu'il est disponible, et la fermeture de boucle. Il exploite directement le LiDAR 3D et l'IMU disponibles, assure une fermeture de boucle native pour corriger la dérive sur de longs trajets, et dispose d'un portage ROS 2 mature.

Au vu de ce panorama comparatif, le choix s'est porté sur LIO-SAM. Clovis ADELER un doctorant du laboratoire travaille sur le diagnostic et l'intégration de nouvelles contraintes liées au GPS dans des architectures multi-robots reposant sur l'optimisation par graphe de facteurs. Le formalisme de résolution par graphe de facteurs, commun à LIOSAM et à ses travaux de recherche, constitue ainsi un point d'ancrage méthodologique. Ce choix s'inscrit enfin dans une logique de continuité : l'objectif à terme de ce stage est de proposer une base capable d'accueillir l'ensemble des briques de recherche développées lors de ce projet, des futures contributions liées aux contraintes GPS en contexte multi-robot ou aux éléments de planification et navigation par exemple.

À noter, en réalité mon choix s'est porté sur LIORF [YJZLuckyBoy, 2022], une variante de LIO-SAM. LIORF reprend l'architecture de LIO-SAM à l'identique, à la différence près qu'il accepte en entrée une IMU 6 axes plutôt que 9 axes. Cette contrainte plus légère facilite grandement les tests et la validation des algorithmes sur des jeux de données publics largement utilisés dans la communauté, tels que KITTI, qui ne fournissent généralement pas de données magnétométriques nécessaires à une IMU 9 axes.

Dans la suite du rapport, après l'explication en détail du fonctionnement de LIOSAM qui suit, le système de SLAM sera désigné par le terme générique « LIO », faisant référence indifféremment à LIO-SAM et à LIORF, bien que ce soit LIORF qui ait été utilisé en pratique.

3.2 LIOSAM

Les quatre contraintes du graphe de facteurs de LIO-SAM

Dans LIO-SAM (figure 3.1), le graphe de facteurs qui relie les états successifs du robot x_i est alimenté par quatre types de contraintes. Le facteur de pré-intégration IMU relie deux états consécutifs x_i et x_{i+1} en intégrant les mesures brutes de l'IMU reçues entre les deux instants correspondants, ce qui fournit une prédiction de mouvement à haute fréquence entre deux poses clés LiDAR.

Le facteur d'odométrie LiDAR relie lui aussi deux états consécutifs, mais à partir du recalage (*scan matching*) du nuage de points courant sur la carte locale construite à partir des dernières poses clés ; il vient corriger la prédiction fournie par l'IMU.

Le facteur GPS, lorsqu'une mesure GPS est disponible, ajoute une contrainte de position absolue sur un état donné, ce qui permet de limiter la dérive à long terme du système en l'absence de fermeture de boucle principalement.

Enfin, le facteur de fermeture de boucle relie deux états non consécutifs, l'état courant et un état passé correspondant à un lieu déjà visité, identifié par recherche de proximité

puis recalage. Ce qui permet de corriger rétroactivement l'ensemble de la trajectoire parcourue entre les deux, contrairement aux trois autres facteurs qui ne contraignent que des états voisins dans le temps.

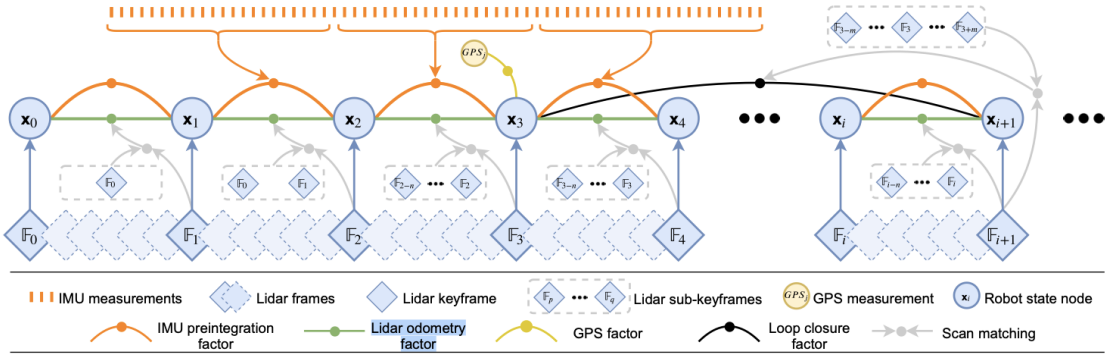


FIGURE 3.1 – Structure du système LIO-SAM : les quatre types de facteurs (préintégration IMU, odométrie LiDAR, GPS, fermeture de boucle) qui relient les états successifs x_i du robot dans le graphe ([Shan et al., 2020]).

Architecture logicielle

Le pipeline logiciel de LIO-SAM se décompose en quatre nœuds ROS principaux, illustrés en figure 3.2. Le nœud `imuPreintegration.cpp` s'abonne aux données IMU brutes et à l'odométrie LiDAR publiée par le nœud d'optimisation ; il assure deux rôles distincts : publier, à la fréquence de l'IMU, une odométrie à haute fréquence obtenue par simple intégration (utilisée comme prédiction par les autres nœuds). Le nœud `imageProjection.cpp` s'abonne au nuage de points, aux données IMU et à cette odométrie ; il calcule une estimée initiale de la transformation, organise le nuage de points reçu et corrige la distorsion du nuage causée par le mouvement du robot pendant l'acquisition du scan (*déskewing*), avant de publier un message `cloud_info`. Le nœud `featureExtraction.cpp` s'abonne à ce message et en extrait les points caractéristiques – arêtes et plans – selon le même principe que LOAM, avant de republier un `cloud_info` enrichi de ces caractéristiques. Le nœud `mapOptimization.cpp`, enfin, s'abonne à ce dernier message ainsi qu'au GPS ; il recale le nuage de points sur la carte locale, en déduit l'odométrie LiDAR, puis exécute l'optimisation globale du graphe de facteurs – combinant les facteurs d'odométrie LiDAR, GPS et de fermeture de boucle – avant de publier l'odométrie LiDAR optimisée, qui est renvoyée en entrée du nœud `imuPreintegration.cpp` pour recaler l'estimation du biais de l'IMU. Cette boucle entre les quatre nœuds constitue le couplage serré caractéristique de LIO-SAM.

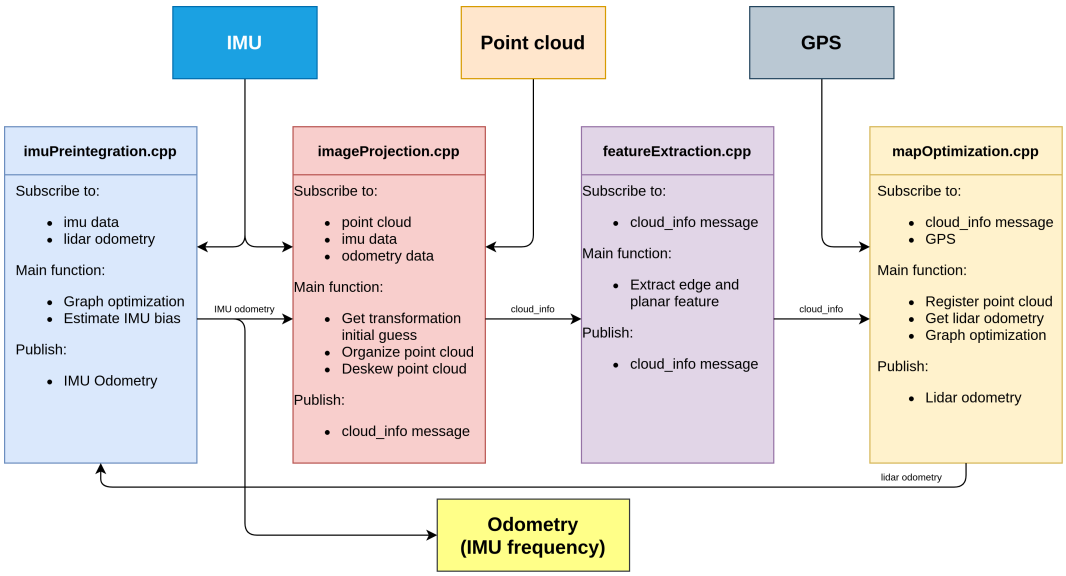


FIGURE 3.2 – Architecture logicielle de LIO-SAM : les quatre nœuds ROS et leurs échanges de messages ([Shan et al., 2020]).

Dans le cadre de ce stage, LIO-SAM a été utilisé et adapté comme brique de cartographie géométrique de base, sur laquelle vient ensuite se greffer la sémantique issue de la détection d'objets par caméra. Les premiers tests de validation ont été menés sur le jeu de données KITTI, largement utilisé comme référence pour évaluer les algorithmes de SLAM en conditions routières réelles.

En complément du système de SLAM, LIO qui ne contient aucune information sémantique, il fallait donc une deuxième brique à imlémenter pour labéliser les points.

3.3 YOLO

Qu'est-ce que YOLO ?

YOLO (*You Only Look Once*) est un réseau de neurones qui analyse une image en une seule passe pour y reconnaître des éléments, contrairement à d'autres approches plus anciennes qui doivent d'abord repérer des zones d'intérêt puis les analyser une par une. Cette simplicité le rend beaucoup plus rapide, ce qui en fait un choix classique pour du traitement en temps réel embarqué sur un robot. YOLO existe en plusieurs tailles de modèle, de *nano* (la plus légère, la moins précise) à *extra-large* (la plus lourde, la plus précise). YOLO a été décliné au fil des années en plusieurs versions (YOLOv5, YOLOv8, YOLO11, YOLO26...), chacune améliorant la précision et la vitesse par rapport à la précédente.

Selon la version et la tâche choisie, YOLO peut donner des résultats différents : une simple boîte autour de chaque objet détecté (détection), un masque de pixels précis pour chaque objet détecté (segmentation d'instance), ou (seulement depuis la version YOLO26) une étiquette pour *chaque pixel de l'image*, y compris le fond (route, végétation, ciel...) : c'est la segmentation *sémantique*.

Pourquoi la version sémantique

Pour ce stage, la version sémantique de YOLO26 (Jocher et al. [2026]) a été retenue plutôt que la détection par boîtes ou la segmentation d'instance, car le besoin est d'at-

tribuer un label sémantique à chaque point LiDAR projeté sur l'image caméra. Avec une simple boîte englobante, un point tombant dans le coin de la boîte peut en réalité appartenir à l'arrière-plan et non à l'objet détecté, ce qui introduit des erreurs d'association. La segmentation d'instance corrige déjà ce premier problème pour les objets détectés, mais elle ne résout pas le second, plus important en pratique : elle ne masque que les objets reconnus comme tels (véhicules, piétons...), et laisse totalement non étiquetées toutes les zones de l'image qui ne sont pas des objets détectés (route, végétation, sol, ciel). Avec la segmentation d'instance seule, la plupart des points resteraient donc sans label. La segmentation sémantique, elle, attribue un label à chaque pixel de l'image, y compris le fond.

Le modèle utilisé est la variante *nano*, choisie pour son faible coût de calcul, un critère important pour un traitement embarqué et temps réel sur robot mobile, couplé à une chaîne de cartographie déjà gourmande en ressources. Ce choix constitue également un point de départ volontairement conservateur plutôt qu'une limite définitive : si la variante *nano* s'avère suffisamment performante pour les besoins du projet, rien n'empêche par la suite de passer à une variante plus lourde (*small*, *medium*...) selon les ressources de calcul réellement disponibles sur le robot cible.

Chapitre 4

Framework pour la cartographie sémantique par fusion LiDAR-caméra

L'approche présentée dans cette section correspond à un framework ROS2, développé pour construire une carte sémantique de l'environnement à partir des deux briques présentées précédemment : la localisation géométrique fournie par LIO et la segmentation sémantique fournie par YOLO. Son rôle est de mettre en correspondance, pour chaque instant, la géométrie 3D issue du LiDAR et l'information sémantique issue de la caméra, puis d'accumuler ces points étiquetés au fil du déplacement du robot pour en déduire, une carte 2D et une carte 3D du terrain parcouru, chacune assortie d'un indice de confiance et d'un indice de traversabilité par zone.

L'objectif de ce développement s'inscrit dans le cadre du projet ANR-SOS : il s'agit de doter chaque robot d'une brique de cartographie sémantique autonome et embarquée, réutilisable à l'identique sur le Ranger et sur le Scout, et suffisamment robuste (gestion des fermetures de boucle, prise en compte d'une carte a priori) pour produire des cartes exploitables ensuite dans une perspective de cartographie coopérative multi-robots.

4.1 Architecture générale

L'algorithme peut se diviser en deux parties distinctes :

- la première composée de la projection des données LiDAR vers la caméra, de l'étiquetage des points via YOLO, et de la transformation dans le repère monde via l'odométrie de LIO ;
- la deuxième partie sert de générateur de carte.

Cette distinction est d'autant plus importante dans un contexte multi-robots, qui sera développé par la suite.

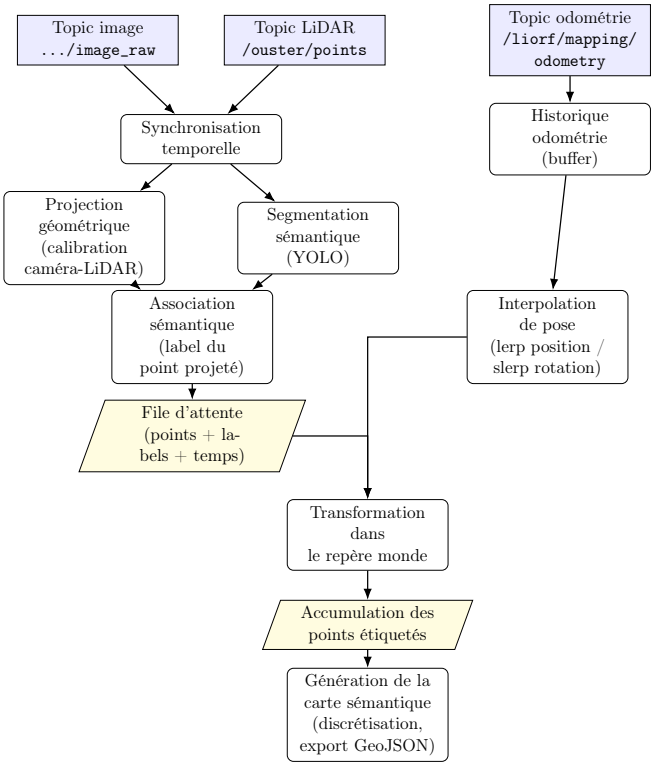


FIGURE 4.1 – Architecture globale du nœud de fusion LiDAR-caméra.

4.2 Acquisition et projection des données sur l'image

4.2.1 Synchronisation temporelle des flux

L'acquisition des données repose sur deux abonnements passifs, mis en place via la bibliothèque `message_filters` : un abonnement au flux image (`sensor_msgs/Image`) et un abonnement au flux LiDAR (`sensor_msgs/PointCloud2`). Ces deux flux ne sont pas publiés exactement au même instant ni à la même fréquence.
La fonction `ApproximateTimeSynchronizer` est donc utilisé pour apparier ces deux topics dont les horodatages sont suffisamment proches (un écart maximal de 50 ms est toléré), et ne déclenche le traitement de fusion que lorsqu'une telle paire est trouvée.

Un second mécanisme de régulation temporelle est appliqué en complément : les capteurs peuvent publier à une fréquence supérieure à ce que la chaîne de traitement peut absorber en temps réel. Un paramètre de fréquence d'acquisition cible est donc défini, et toute paire image/LiDAR reçue avant l'écoulement de l'intervalle minimal correspondant est simplement ignorée, plutôt que d'accumuler un retard croissant.

4.2.2 Projection géométrique des points LiDAR sur l'image

Pour chaque paire synchronisée, le nuage de points LiDAR est d'abord lu et converti en coordonnées homogènes. Les points sont ensuite projetés dans le plan image en deux temps : un premier produit matriciel les fait passer du repère du LiDAR au repère optique de la caméra (calibration extrinsèque), puis un second produit matriciel les projette dans

le plan image (calibration intrinsèque). Le résultat de cette double transformation fournit, pour chaque point, une coordonnée pixel (u, v) ainsi qu'une profondeur associée.

Tous les points ne sont cependant pas exploitables : plusieurs filtres successifs sont appliqués pour ne conserver que les points réellement utiles à la fusion. Un premier filtre élimine les points dont la profondeur projetée est négative ou nulle, c'est-à-dire les points situés derrière le plan de la caméra. Un second filtre ne conserve que les points dont la coordonnée pixel (u, v) tombe effectivement dans les limites de la résolution de l'image, c'est-à-dire dans le champ de vision réel de la caméra. Un troisième filtre élimine enfin les points dont la distance dépasse une portée maximale paramétrable, afin d'écarter les mesures LiDAR les plus lointaines et les moins fiables pour l'association sémantique.

Matrices de calibration caméra-LiDAR

La matrice intrinsèque, obtenue à partir du topic d'information caméra, dépend des paramètres internes du capteur et de la résolution d'image. La matrice extrinsèque, elle, encode la transformation rigide (rotation et translation) entre le repère du LiDAR et le repère optique de la caméra ; elle diffère sensiblement d'un robot à l'autre puisqu'elle dépend directement du positionnement physique des deux capteurs l'un par rapport à l'autre. Ces deux matrices sont déterminées une fois pour chaque robot et chargées comme paramètres au démarrage du nœud.

4.2.3 Segmentation sémantique via YOLO et association aux points

En parallèle de la projection géométrique, l'image est passée dans le modèle YOLO en segmentation sémantique (variante *nano*), qui produit un masque associant à chaque pixel de l'image un label. L'association sémantique consiste alors simplement à lire, pour chaque point LiDAR valide après filtrage, la valeur de ce masque au pixel (u, v) correspondant, puis à convertir cet identifiant en nom de classe lisible. C'est cette étape et non la projection géométrique seule qui constitue la fusion à proprement parler : chaque point conserve sa position 3D précise tout en héritant d'un label sémantique.

Un affichage temps réel est également mis en place à titre de contrôle : le masque sémantique est superposé à l'image caméra, et les points LiDAR projetés y sont dessinés, colorés selon leur profondeur, avec une légende des classes détectées. Cet affichage ne participe pas au calcul de la carte finale mais permet de vérifier visuellement, en direct, la qualité de la calibration et de l'association point-pixel.

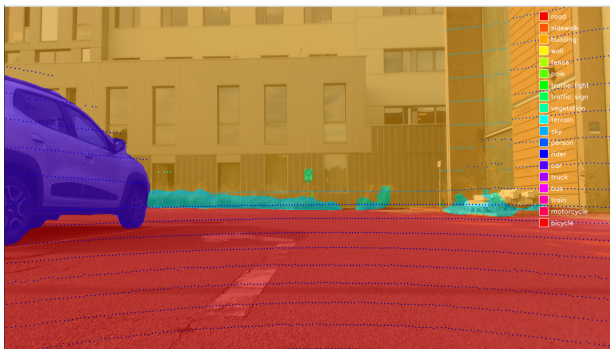


FIGURE 4.2 – Image OpenCV : masque sémantique YOLO et points LiDAR 3D

Sur la figure 4.2, la légende indique le nom des labels associés à chaque couleur du masque sémantique, tandis que la couleur des points LiDAR projetés diffère selon leur

portée.

4.3 Passage dans le repère monde

Pour transformer les points LiDAR dans le repère monde, il est nécessaire de connaître la pose du robot à l'instant considéré. Deux approches ont été envisagées pour l'obtenir : s'appuyer sur l'arbre de transformations ROS (TF), ou s'abonner directement au topic d'odométrie publié par LIO.

C'est cette seconde solution qui a été retenue : le nœud s'abonne directement au topic d'odométrie et reconstitue lui-même, par interpolation, la pose correspondant à l'horodatage exact de chaque trame LiDAR-caméra traitée. Ce choix s'explique par le coût de calcul de l'approche TF : chaque recherche de transformée nécessite de parcourir l'arbre de repères (`odom` → `base_link` → `lidar_link`, etc.)

4.3.1 Synchronisation temporelle

Cette synchronisation est distincte de celle décrite précédemment entre l'image et le LiDAR : il s'agit ici de faire correspondre l'horodatage d'une trame déjà projetée et étiquetée avec le flux d'odométrie, qui est publié de façon indépendante et n'arrive pas nécessairement au même rythme. Une trame n'est traitée que si l'historique d'odométrie couvre déjà son horodatage ; dans le cas contraire, elle est laissée en attente jusqu'à ce qu'une odométrie plus récente soit reçue.

4.3.2 Queue de fonctionnement

Les trames traitées (points projetés, labels, horodatage) sont stockées dans une file d'attente de taille bornée. Cette file est parcourue dans l'ordre d'arrivée : tant que la trame la plus ancienne peut être associée à une odométrie interpolable, elle est transformée puis retirée de la file, ce qui garantit à la fois l'ordre temporel du traitement et une consommation mémoire maîtrisée même en cas de léger retard de l'odométrie par rapport aux données caméra/LiDAR.

4.3.3 Interpolation de l'odométrie

À chaque nouvelle réception, le message d'odométrie est ajouté à un historique. Pour retrouver la pose du robot à un horodatage cible, les deux messages d'odométrie encadrant cet horodatage sont recherchés dans l'historique, puis une interpolation est appliquée séparément sur la position (interpolation linéaire) et sur l'orientation (interpolation sphérique du quaternion) (avec la bibliothèque SciPy voir Virtanen et al. [2020]). La pose ainsi reconstruite est ensuite utilisée pour transformer les points de la trame depuis le repère du LiDAR vers le repère monde.

4.4 Parseur et générateur de carte

Une fois les points accumulés dans le repère monde, un ensemble de fonctions dites « hors ligne », déclenchées à l'arrêt du nœud, prennent le relais pour transformer ce nuage de points étiquetés en une carte exploitable. À noter que ces fonctions, aujourd'hui déclenchées uniquement à l'arrêt du nœud, devront à terme être exécutées périodiquement

(toutes les x secondes) plutôt qu'une seule fois en fin d'exécution, dans la mesure où le système est amené à fonctionner en temps réel et sur de longues durées ; ce point sera détaillé dans la partie consacrée au multi-robots.

Génération de la carte 2D

Les points accumulés sont d'abord regroupés en une grille de cases carrées, à l'aide de la bibliothèque GeoPandas. Pour chaque case, le label dominant est déterminé (le label le plus représenté parmi les points qui y tombent), le temps d'acquisition (min et max des labels domiannts) ainsi que le pourcentage de points portant ce label dominant, utilisé comme indicateur de confiance sur la case. Le résultat est exporté au format GeoJSON (une case = une entité, avec son centre, son label dominant et ce pourcentage).

Génération de la carte 3D (voxelisation)

Le principe est similaire en trois dimensions, mais la démarche diffère techniquement : les opérations géométriques utilisées en 2D (jointure spatiale, polygones) ne sont pas directement applicables à un espace 3D. Chaque point est donc indexé selon trois indices entiers de voxel (calculés à partir de sa position et du pas de discrétisation choisi en x, y et z), puis les points sont regroupés par voxel pour en extraire le label dominant, le temps d'acquisition et le pourcentage associé. Le résultat est exporté au format GeoJSON, chaque voxel étant représenté par son point central, et visualisé sous forme d'un nuage de points 3D coloré par classe.

Confiance et traversabilité

Deux informations supplémentaires sont calculées lors de l'agrégation par case ou par voxel. D'une part, une confiance liée à la portée de mesure est associée à chaque point dès son intégration dans le nuage accumulé, décroissante avec la distance au LiDAR au moment de l'acquisition (un point mesuré proche du capteur est considéré plus fiable qu'un point mesuré à la portée maximale). D'autre part, un indice de traversabilité est déduit du label dominant de chaque case ou voxel, par une table de correspondance associant à certains labels (par exemple une route) une valeur de traversabilité élevée ; cette table reste, à ce stade, restreinte à quelques labels et constitue une piste d'enrichissement pour la suite du projet par exemple en y ajoutant la prise en compte de l'élévation.

4.5 Tests et vérifications

Le jeu de données KITTI [Geiger et al., 2013] a été utilisé pour valider le fonctionnement du framework *.

4.5.1 Installation et configuration des données KITTI

Pour intégrer les données KITTI au framework, plusieurs approches ont été envisagées. La première repose sur un nœud Python, le *kitti publisher node*, qui republie en temps réel les données KITTI sous forme de topics ROS en les lisant directement depuis les fichiers du jeu de données. Ce script, disponible sur la documentation officielle de KITTI

*. Séquences « city » du jeu de données brutes KITTI : https://www.cvlibs.net/datasets/kitti/raw_data.php?type=city

et développé par des utilisateurs tiers, a posé des problèmes d'horodatage ainsi qu'une incompatibilité de type sur le topic GPS.

La seconde approche, *kitti2bag*, consiste à convertir l'intégralité des données KITTI en un unique rosbag. Bien que ce script n'existe qu'en version ROS1, il présente l'avantage de ne nécessiter la conversion qu'une seule fois : le jeu de données obtenu est plus portable, puisqu'il n'est plus nécessaire de conserver les données KITTI brutes une fois le bag généré. Ce script a donc été porté vers ROS2 (une version ROS1 équivalente existe également dans le dépôt de LIO-SAM).

Une attention particulière doit être portée à l'organisation des données KITTI, notamment lorsqu'elles sont téléchargées séparément (LiDAR, caméra, etc.), chaque type de donnée devant être placé dans l'arborescence attendue par le script de conversion. Il est également nécessaire de télécharger les données de calibration correspondantes, détaillées ci-dessous.

Capteurs disponibles depuis KITTI

À noter que le jeu de données KITTI utilisé repose sur un LiDAR Velodyne 64 couches, une caméra couleur 2D (1242 * 375 pixels), un IMU 6 axes et un GPS.

Lors de la configuration de LIO, trois points se sont révélés particulièrement importants à vérifier.

Matrices de calibration

Les matrices de calibration correspondent aux matrices de transformation entre le LiDAR et l'IMU. En effet, LIO couple fortement le LiDAR et l'IMU : il a donc besoin de ces transformations, à savoir une matrice de translation et une matrice de rotation, que l'on trouve dans le fichier de configuration des données KITTI.

Champ `time`

Le champ `time` représente le décalage temporel de chaque point du LiDAR par rapport au début d'un scan. LIO l'utilise pour corriger le *deskewing*, c'est-à-dire la distorsion du nuage de points créée par la rotation du capteur pendant l'acquisition.

Les données brutes KITTI ne l'incluent pas dans le nuage de points. Ce champ n'a pas été reconstruit sur ce jeu de données au cours du stage : son ajout n'est pas indispensable au fonctionnement du framework, mais constitue une piste d'amélioration à approfondir.

Champ `ring`

Le troisième point de vigilance concerne le champ `ring`, qui indique, pour chaque point, le numéro de la nappe laser (le canal vertical) dont il est issu. Contrairement au champ `time`, ce champ est indispensable au bon fonctionnement de LIO, qui l'utilise pour organiser le nuage de points par nappe.

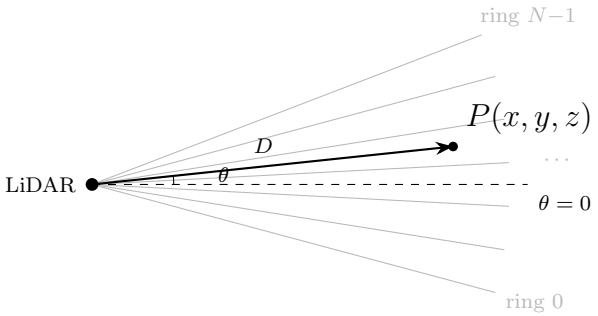


FIGURE 4.3 – Principe du calcul du numéro de nappe (`ring`) à partir de l'angle d'élévation θ : chaque nappe du LiDAR correspond à un angle d'élévation fixe, et le point mesuré P est rattaché à la nappe dont l'angle est le plus proche de θ. D est la distance du point au LiDAR, utilisée dans le calcul de θ.

Ce champ n'étant pas fourni tel quel par les données brutes KITTI, il est recalculé dans le nœud Python chargé de la conversion du bag. Pour chaque point de coordonnées (x, y, z) dans le repère du LiDAR, on calcule d'abord sa distance D au capteur, puis son angle d'élévation θ :

$$D = \sqrt{x^2 + y^2 + z^2}, \qquad \theta = \arcsin\left(\frac{z}{D}\right)$$

Le numéro de nappe est ensuite obtenu en ramenant θ dans l'intervalle $[0, N]$ (avec $N = 64$ nappes par exemple) entre l'angle d'élévation minimal $\theta_{\min}$ et maximal $\theta_{\max}$ du champ de vision vertical du capteur ($\theta_{\min} = -24.8°$ et $\theta_{\max} = 2°$ pour le Velodyne utilisé).

$$\text{ring} = \frac{\theta - \theta_{\min}}{\theta_{\max} - \theta_{\min}} \times N$$

Cette formule revient à découper le champ de vision vertical en N tranches angulaires égales et à identifier, pour chaque point, la tranche dans laquelle il tombe (figure 4.3). Elle suppose donc un espacement angulaire uniforme entre les nappes, ce qui reste également une approximation.

4.5.2 Validation de LIO sur les données KITTI

La deuxième étape a consisté à valider le fonctionnement de LIO sur ces données.

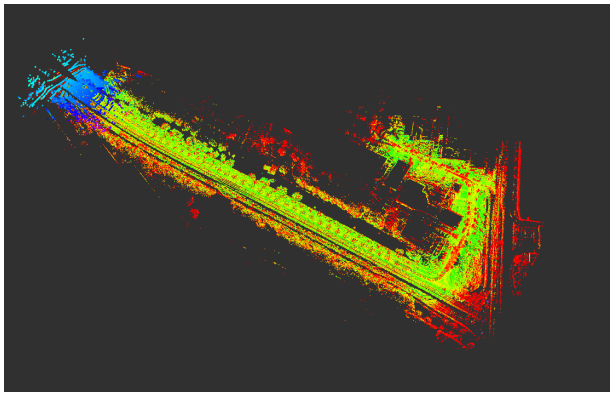


FIGURE 4.4 – Scan du bag KITTI traité par LIO.

La figure 4.4 présente le scan du bag KITTI utilisé, d'une durée d'un peu plus de 60 secondes. À noter que le véhicule y évolue en environnement urbain et y atteint plus de 50 km/h sur la grande ligne droite, une vitesse nettement supérieure à celle attendue pour nos futurs robots.

4.5.3 Validation du framework complet

Une fois le bag KITTI et LIO validés, le framework complet a pu être testé dans son ensemble.

Le modèle YOLO utilisé est bien entraîné pour ce type d'environnement urbain ; la version *nano* a toutefois dû être retenue, ce choix étant quasiment imposé par les capacités de calcul limitées du PC.

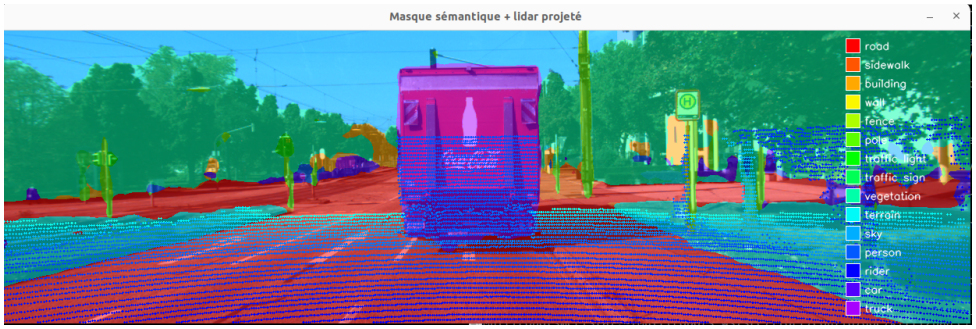
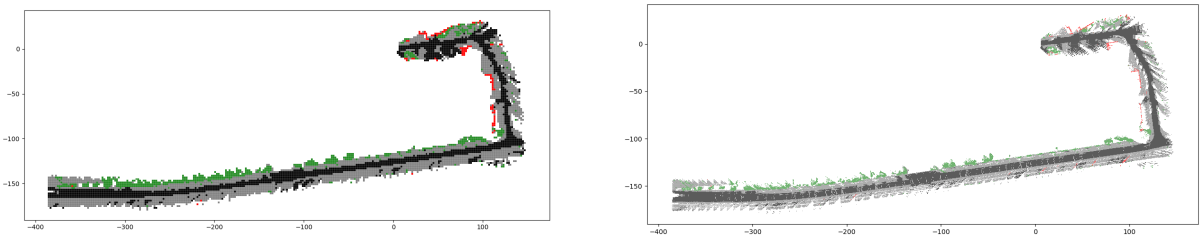


FIGURE 4.5 – Projection en temps réel des points LiDAR sur l'image 2D.

La figure 4.5 est une capture d'écran de la vidéo OpenCV affichée en temps réel, permettant de valider visuellement la bonne calibration des points 3D du LiDAR projetés sur l'image 2D et des masques sémantiques réalisés par YOLO.

Cartes générées

Ces différentes cartes sont une représentation du fichier GeoJSON produit par le framework, la sortie finale attendue, contenant les voxels avec leurs coordonnées et leurs labels, comme décrit précédemment. Concernant les couleurs, seuls cinq labels ont été distingués sur ces cartes : les bâtiments en rouge, la végétation en vert, la route en noir et l'eau en bleu ; le gris regroupe le sol hors route ainsi que tous les autres labels non distingués.

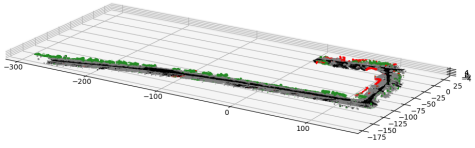


(a) Pas de 1 m.

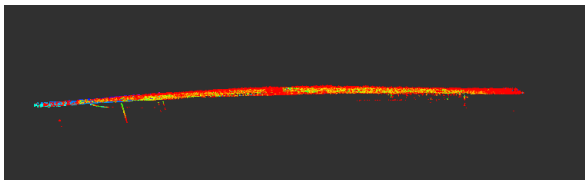
(b) Pas de 0.5 m.

FIGURE 4.6 – Carte sémantique 2D générée à partir du bag KITTI, pour deux pas de discrétisation.

La figure 4.6 présente la carte générée par le parseur, discrétisée avec un pas de 1 m (figure 4.6a) puis de 0.5 m (figure 4.6b). Bien que cette carte soit en réalité tridimensionnelle, toutes les valeurs y sont aplaties en deux dimensions pour cette représentation.



(a) Carte générée 3D générée à partir du bag KITTI.



(b) Carte native produite par LIO.

FIGURE 4.7 – Carte sémantique en trois dimensions générée à partir du bag KITTI, comparée à la carte native produite par LIO.

La figure 4.7 présente cette même carte en trois dimensions (figure 4.7a). On y remarque une légère courbure, que l'on retrouve également sur la carte produite nativement par LIO (figure 4.7b).

Ce jeu de données a ainsi permis de valider le bon fonctionnement global du framework.

Chapitre 5

Loop closure

Pour rappel, une fermeture de boucle (*loop closure*) consiste à reconnaître qu'un robot repasse par un endroit déjà visité, ce qui permet de corriger l'accumulation progressive d'erreur (la dérive) sur l'ensemble de la trajectoire estimée depuis le début du parcours, plutôt que sur la seule portion la plus récente. LIO réalise des corrections globales à partir de cette détection de fermeture de boucle ; l'enjeu est donc de répercuter cette correction sur notre carte sémantique globale également.

5.1 Détection de loop closure depuis LIO

La détection de la fermeture de boucle en elle-même n'est pas réalisée par notre framework, mais en amont, par LIO, qui se contente ensuite de publier ses résultats sur les topics écoutés par notre node (contraintes et trajectoire corrigée).

Le principe de détection utilisé par LIO est le suivant : à chaque nouvelle pose clé (*keyframe*), une recherche par arbre en 3 dimensions est effectuée parmi l'ensemble des poses clés déjà enregistrées, afin de trouver celles situées à proximité spatiale de la pose courante. Les candidates trop proches dans le temps de la pose courante sont écartées, de façon à ne pas comparer la trajectoire avec elle-même quelques instants plus tôt.

Pour chaque candidate restante, une sous-carte locale est construite en agrégeant plusieurs poses clés voisines autour de la pose courante d'une part, et autour de la candidate d'autre part, puis les deux sous-cartes sont recalées l'une sur l'autre par ICP (*Iterative Closest Point*). La fermeture de boucle n'est finalement acceptée – et ajoutée comme nouvelle contrainte au graphe de facteurs servant à l'optimisation globale de la trajectoire – que si le score de qualité de ce recalage ICP passe sous un seuil donné.

5.1.1 Paramètres modifiables dans LIO

Plusieurs paramètres de LIO permettent d'ajuster ce comportement :

- `loopClosureEnableFlag` : active ou désactive entièrement la détection de fermeture de boucle.
- `loopClosureFrequency` (1 Hz par défaut) : fréquence à laquelle une tentative de détection est effectuée.
- `historyKeyframeSearchRadius` (15 m par défaut) : rayon de recherche spatiale autour de la pose courante pour identifier des poses clés candidates.

— `historyKeyframeSearchTimeDiff` (30 s par défaut) : écart temporel minimal requis entre la pose courante et une candidate pour qu'elle soit retenue, ce qui évite de fermer une boucle avec une portion de trajectoire trop récente.
— `historyKeyframeSearchNum` (25 par défaut) : nombre de poses clés voisines agrégées de part et d'autre pour construire les deux sous-cartes comparées par ICP.
— `historyKeyframeFitnessScore` (0.3 par défaut) : seuil de score de qualité ICP endessous duquel la contrainte est acceptée – plus la valeur est basse, plus l'exigence de recalage est stricte.

C'est le réglage de ces paramètres, en particulier `historyKeyframeSearchRadius` et `historyKeyframeFitnessScore`, qui détermine en pratique la sensibilité de la détection : un rayon trop grand ou un seuil de fitness trop permissif favorisent les fausses détections, tandis que des valeurs trop strictes risquent de manquer de véritables fermetures de boucle.

5.2 Évolution de l'architecture

La gestion des fermetures de boucle vient se greffer sur la chaîne de fusion décrite précédemment, sous la forme de deux abonnements supplémentaires : l'un aux contraintes de fermeture de boucle publiées par LIO, l'autre à la trajectoire complète corrigée.

La figure 5.1 illustre cette évolution de l'architecture par rapport au pipeline de base présenté précédemment (en pointillé), avec l'ajout de la gestion des fermetures de boucle (en trait plein).

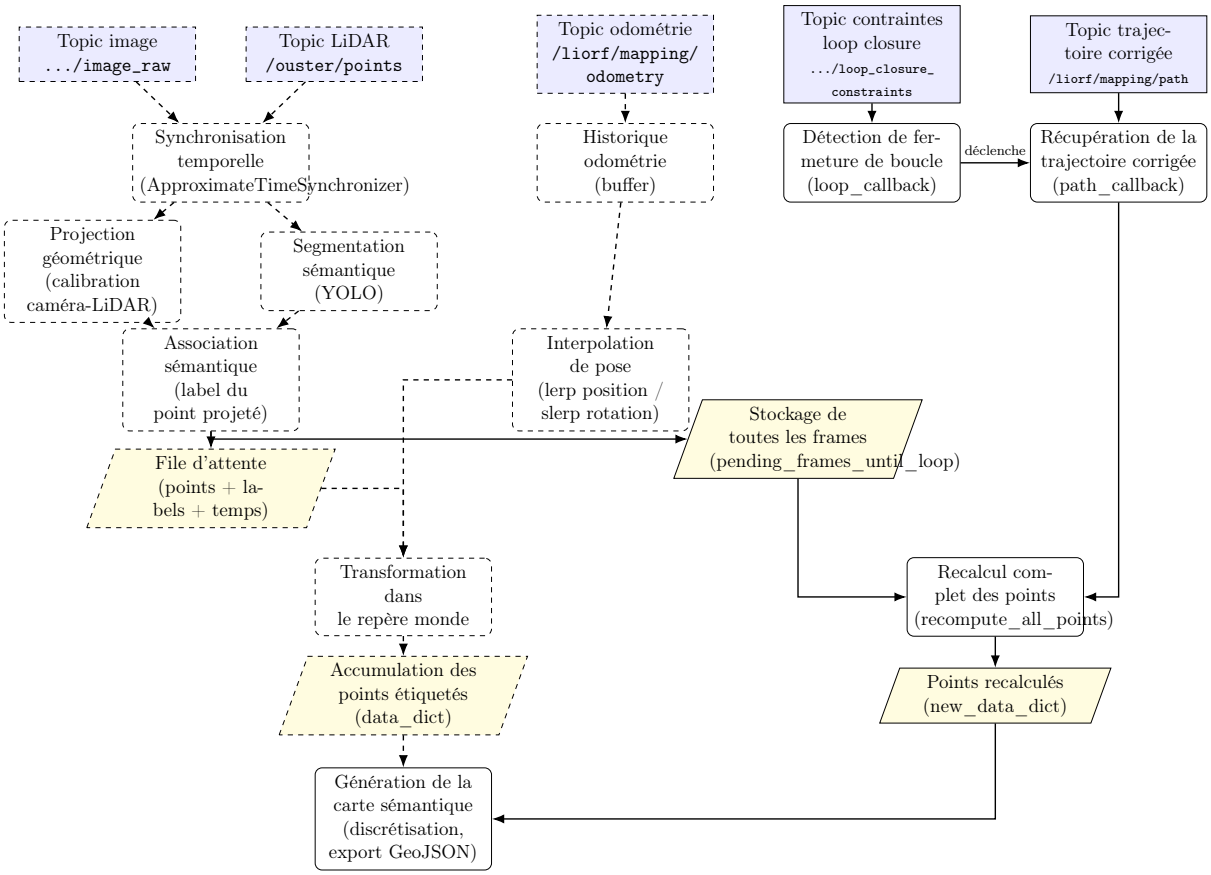


FIGURE 5.1 – Architecture globale du nœud de fusion, avec ajout de la gestion des fermetures de boucle (en trait plein ; pipeline de base en pointillé)

5.2.1 Détection d'une loop closure

Une fermeture de boucle est détectée en observant l'évolution du nombre de contraintes publiées : toute augmentation de ce nombre signale qu'une nouvelle fermeture de boucle vient d'être intégrée par LIO.

5.2.2 Mise à jour des points

Une fois une fermeture de boucle détectée, l'ensemble des points déjà accumulés doit être mis à jour pour tenir compte de la trajectoire corrigée. Cette mise à jour repose sur quatre éléments : un stockage complémentaire de l'historique des trames, la file d'attente de traitement courante qui continue de fonctionner sans interruption, un mécanisme d'interpolation appliqué cette fois à la trajectoire corrigée, et enfin le recalcul complet des points qui en résulte.

Stockage des points

L'ensemble des frames traitées depuis le début de l'exécution est conservé, dans un second stockage dédié (`pending_frames_until_loop`). Chaque frame y est gardée sous sa forme brute (points LiDAR filtrés et labels associés), avant transformation dans le repère monde. Ce qui permet de retransformer l'intégralité de l'historique si il y a une fermeture de boucle.

Interpolations

Le recalcul complet réutilise le même principe d'interpolation que le fonctionnement courant, mais appliqué cette fois à la trajectoire corrigée plutôt qu'à l'historique brut de l'odométrie : pour chaque frame stockée, les deux poses de la trajectoire corrigée encadrant son horodatage sont recherchées, puis interpolées pour obtenir la pose exacte à appliquer. Cette interpolation est d'autant plus nécessaire car la trajectoire corrigée ne contient que les poses des *key frames* du robot, ce qui représente au final beaucoup moins de poses disponibles qu'avec l'odométrie.

Recalcul des points

Dès qu'une fermeture de boucle est détectée, la trajectoire corrigée correspondante est récupérée. Cette trajectoire déclenche alors un recalcul complet : contrairement au fonctionnement normal, où seules les nouvelles trames sont transformées et accumulées au fil de l'eau, toutes les trames reçues depuis le début de l'exécution sont conservées en parallèle dans un second stockage dédié, précisément pour permettre ce recalcul. Chacune de ces trames est alors retransformée dans le repère monde, non plus à partir de l'odométrie brute interpolée, mais à partir de la trajectoire corrigée elle-même interpolée de la même façon (interpolation linéaire de la position, interpolation sphérique de l'orientation). Ce recalcul produit un nouveau nuage de points, qui remplace intégralement l'ancien nuage accumulé pour la génération de la carte finale.

5.3 Tests et validations

5.3.1 Test de la fermeture de boucle sur les données KITTI

Le bag KITTI utilisé précédemment ne correspond pas à un tour complet : le robot ne repasse pas par son point de départ. Pour tout de même tester le bon fonctionnement de la détection de fermeture de boucle et du recalcul des points associé, il suffit de modifier les paramètres de *loop closure* de LIO afin de la déclencher dans des conditions plus larges, par exemple en raccourcissant fortement le délai minimal avant détection, de sorte qu'une fermeture de boucle soit détectée sur la trajectoire qui vient d'être parcourue. Ce délai est normalement fixé précisément pour éviter ce cas de figure (voir section précédente). Ce réglage n'a donc ici pour seul but que de valider le bon fonctionnement du framework et sa communication avec LIO, et non de représenter un cas d'usage réaliste.

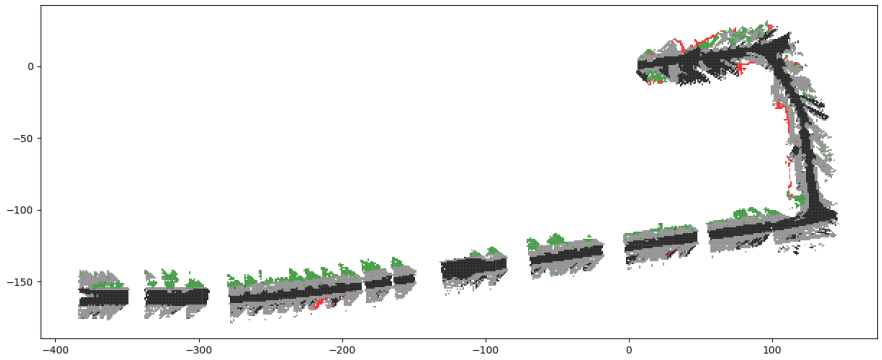


FIGURE 5.2 – Carte 2D générée à partir du bag KITTI, fermeture de boucle activée.

Sur la figure 5.2, on observe des pertes de données répétées à certains endroits de la carte. Ceci est principalement dû à l'activation de la fermeture de boucle : le recalcul des points demande en effet beaucoup de ressources, d'autant plus que le temps avance, puisque les données à recalculer s'accumulent.

Ce phénomène est par ailleurs aggravé par la vitesse du véhicule, qui peut atteindre plus de 50 km/h ce qui suffit à faire perdre des données. On remarque d'ailleurs que les pertes de données commencent exactement au moment où le véhicule prend beaucoup de vitesse, ce qui correspond à la grande ligne droite du parcours.

On remarque qu'au début du bag, aucune perte de données n'est observée ; on peut donc en conclure qu'il ne faudra pas dépasser une certaine vitesse avec nos robots, ni un certain délai avant de transférer la carte, afin de ne pas accumuler trop de données à recalculer en cas de fermeture de boucle.

Chapitre 6

Carte à-priori

Après avoir présenté la chaîne de fusion sémantique LiDAR-caméra permettant au robot de construire sa propre carte de l'environnement et l'ajout de la loop closure dans le framework, ce chapitre s'intéresse à la construction d'une carte dite « a priori », c'est-à-dire une carte du terrain élaborée en amont, à partir de données déjà existantes (OpenStreetMap), plutôt que produite par le robot lui-même. Cette carte a priori répond à deux besoins qui viennent compléter la carte issue de la fusion :

— servir de référence de navigation, disponible dès le démarrage du robot et indépendamment de ce qu'il a lui-même déjà cartographié,
— servir de vérité de terrain à laquelle comparer la carte sémantique effectivement produite par le nœud de fusion présenté précédemment

C'est également cette carte a priori qui sera, à terme, réintégrée dans le processus de cartographie lui-même comme source d'information complémentaire.

6.1 Prélèvement de la carte

La carte a priori de l'environnement a été construite à partir de données OpenStreetMap (OpenStreetMap contributors [2026]), extraites via l'outil *Overpass Turbo* (Raifer [2013]) au format GeoJSON, en sélectionnant les éléments pertinents pour le contexte du projet.

6.2 Discrétisation de la carte a priori

Afin de pouvoir comparer la carte a priori aux cases explorées par les robots, celle-ci doit être discrétisée sous forme de grille de cases homogènes avec la représentation utilisée pour l'exploration.

Cette discrétisation a été réalisée à l'aide de la bibliothèque **GeoPandas** (Fleischmann et al. [2026]), qui facilite la manipulation de données géographiques. Notamment, par une opération de jointure spatiale (fonction `sjoin`) entre la grille générée et les polygones GeoJSON de la carte a priori, produisant en sortie un fichier GeoJSON simplifié.

Un problème est apparu lors de cette étape : lorsqu'une case de la grille recouvre partiellement plusieurs polygones portant des labels différents (par exemple une partie « bâtiment » et une partie « forêt »), l'opération de jointure spatiale crée deux entrées distinctes pour la même case. Deux pistes de solution ont été envisagées :

— **réduire le pas de discrétisation** : cela diminue le nombre de cas de doublon, sans supprimer le problème pour autant (une case peut, quelle que soit sa taille,

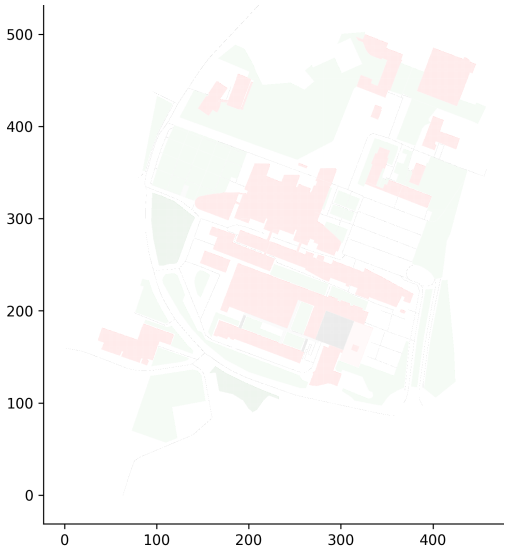
chevaucher une frontière entre deux zones).

— **établir une priorité entre labels** (par exemple, un obstacle « bâtiment » prévaut toujours sur une zone « libre ») : cette solution résout le doublon mais entraîne une perte d'information, la case perdant la trace de sa composition réelle.

En combinant ces deux approches, on obtient une carte 2D discrétisée relativement précise. De plus, on peut se permettre de choisir un pas de discrétisation assez fin, car la carte a priori ne sera chargée qu'une seule fois, au lancement des robots : le facteur limitant la taille des cases est en réalité la carte construite en temps réel par le robot lui-même, et non la carte a priori.



(a) Zone prélevée depuis Overpass turbo.



(b) Carte a priori discrétisée obtenue (rouge : bâtiment et vert : végétation).

FIGURE 6.1 – Zone extraite depuis Overpass turbo et carte a priori discrétisée correspondante.

6.2.1 Classification sémantique des cellules

La priorité entre labels est appliquée par une fonction de classification qui parcourt les tags OpenStreetMap présents sur les polygones intersectant chaque case, selon un ordre de priorité fixe. Par exemple, si une case contient à la fois les labels *building* et *road*, la classe retenue sera *building*, par précaution (un obstacle prévalant sur une zone franchissable).

Cette étape de classification effectue également une simplification et une normalisation des labels. En effet, OpenStreetMap propose une multitude de valeurs possibles pour un même type d'élément, dont certains détails ne sont pas essentiels pour ce projet ; de plus, la forme exacte des tags dans le GeoJSON exporté via Overpass Turbo n'est pas toujours homogène (par exemple, un bâtiment est parfois indiqué par `building=yes`). La fonction de classification regroupe donc ces différentes variantes sous un label unique et normalisé (par exemple *building*) dans le GeoJSON final exporté.

6.3 Format et exportation

Le format de sortie retenu est une ligne une *feature* GeoJSON par case, contenant les coordonnées du centre de la case (les coordonnées du polygone de la case restant également disponibles si souhaité) ainsi que le label qui lui est associé. Chaque case classifiée

est ainsi exportée au format GeoJSON avec son identifiant, ses coordonnées et son label, comme illustré (voir Annexe) issu du fichier `export_grid.geojson`.

Cependant, à noter que l'absence de prise en compte de l'élévation (altimétrie) dans la carte à piori représente une limite à prendre en comtpe, notamment dans le contexte de drone aérien.

6.4 Implementation de la carte dans l'algorithme

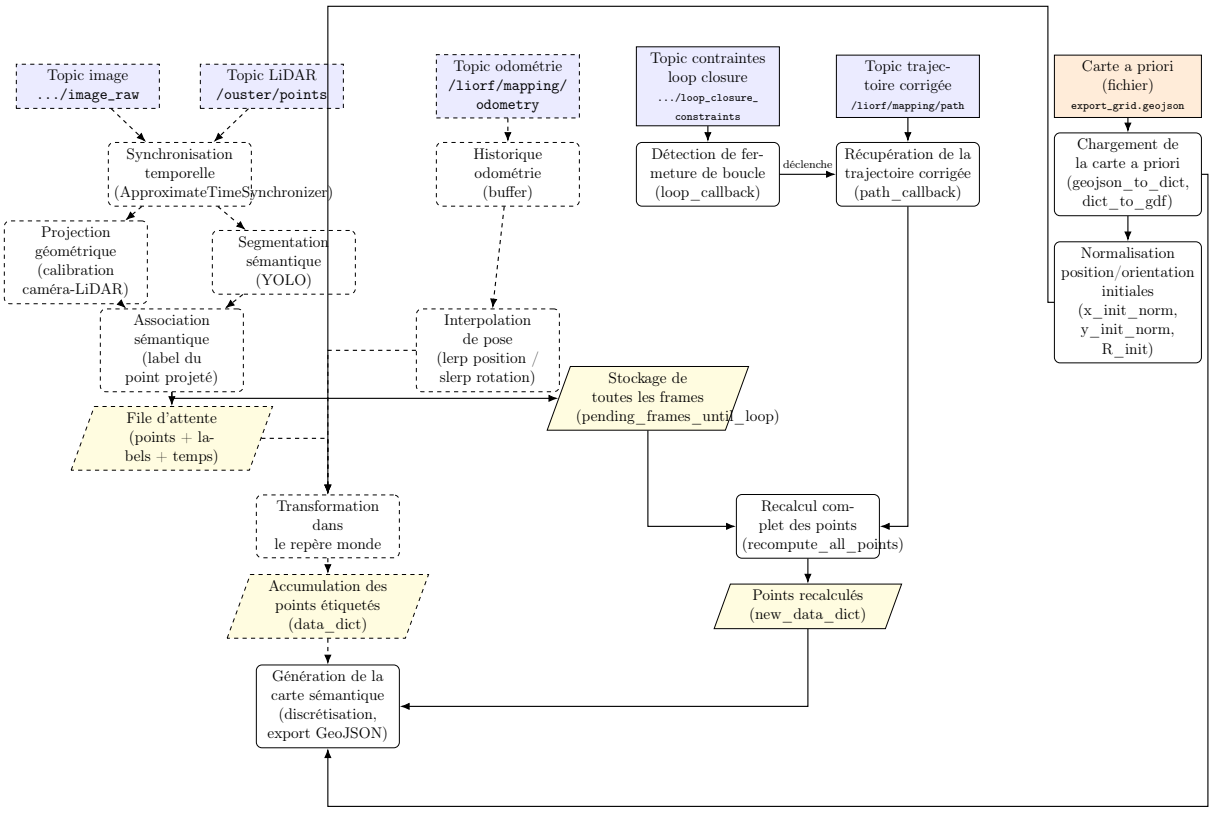


FIGURE 6.2 – Architecture globale du nœud de fusion. Pipeline de base en pointillé ; ajout loop closure en trait plein (noir) ; ajout carte a priori en trait plein (orange)

Une fois la carte a priori discrétisée disponible sous forme du fichier `export_grid.geojson`, son intégration au sein du nœud de fusion se fait au démarrage du nœud, en plusieurs étapes (illustrées en orange sur la figure 6.2) :

- **Chargement.** Le fichier GeoJSON est relu et transformé en un dictionnaire interne associant à chaque identifiant ses coordonnées et son label, puis reconverti en `GeoDataFrame` en reconstruisant, pour chaque entité, un polygone carré de la taille d'une case, centré sur ses coordonnées.
- **Normalisation du repère.** Les coordonnées de la carte a priori sont exprimées dans le système de projection Lambert-93 (des valeurs élevées, peu maniables en pratique). L'origine minimale de la carte a priori, calculée une fois en amont, est donc soustraite à toutes les coordonnées manipulées par le nœud, afin de ramener l'ensemble des calculs dans un repère local proche de zéro.
- **Position et orientation initiales du robot.** La position et le cap initial du robot dans le repère de la carte a priori sont déterminés au préalable et renseignés

en dur au démarrage du nœud. Le cap sert à construire une matrice de rotation autour de l'axe vertical, tandis que la position, une fois normalisée dans le même repère que la carte a priori, fournit un décalage à appliquer en x et en y.

— **Application dans la chaîne de transformation.** Ces deux éléments sont utilisés dans la fonction qui transforme les points du repère LiDAR vers le repère monde : après la transformation classique déduite de l'odométrie de LIO (rotation et translation), les points subissent une rotation supplémentaire par cette matrice, afin d'aligner l'orientation du repère odométrique sur celle de la carte a priori, puis reçoivent le décalage de position calculé précédemment. Les points du nuage sémantique accumulé se retrouvent ainsi exprimés dans le même repère que la carte a priori, ce qui permet de les superposer directement.

Il est à noter que, dans l'état actuel de l'implémentation, la carte a priori n'est utilisée que comme fond de carte affiché en arrière-plan lors de la génération des visualisations : elle n'est pas incluse dans le fichier GeoJSON produit en sortie du framework, qui ne contient que les cases issues de la carte construite en ligne par le robot. Son intégration directe serait pourtant simple à réaliser d'un point de vue technique, puisqu'elle est déjà disponible sous la forme d'un `GeoDataFrame`, discrétisée selon le même format que la carte produite par le robot, et fiable aussi bien géométriquement que sémantiquement. Ce choix n'a cependant pas été fait à ce stade, car il nécessiterait de définir au préalable une méthode de fusion entre la carte construite par le robot et la carte a priori.

Chapitre 7

Extension multi-robots

La chaîne de fusion présentée jusqu'ici a été pensée et développée pour un seul robot. Dans le cadre du projet ANR-SOS, l'objectif final est cependant de produire une carte sémantique unique, partagée, construite à partir des observations de plusieurs robots évoluant simultanément sur le même terrain. Cette section décrit les modifications apportées au code pour passer d'une architecture mono-robot, où un seul nœud effectuait à la fois la fusion et la cartographie, à une architecture multi-robots où ces deux rôles sont séparés et communiquent par topics ROS2.

7.1 Nouvelle architecture

7.1.1 Séparation en deux nœuds

Le nœud unique décrit précédemment a été scindé en deux nœuds distincts. Le premier, `PerceptionNode`, est instancié une fois par robot et reprend l'essentiel de la chaîne de fusion déjà présentée : synchronisation image/LiDAR, projection géométrique avec la calibration propre à ce robot, segmentation sémantique YOLO, association point-pixel, interpolation de l'odométrie et gestion de la fermeture de boucle. La différence essentielle est qu'il ne construit plus lui-même de carte : une fois un point transformé dans le repère monde, il est publié sur un topic dédié à ce robot plutôt qu'accumulé localement.

Le second nœud, `MapFusionNode`, est instancié une seule fois pour l'ensemble du système. Il s'abonne aux topics publiés par chacun des robots déclarés (paramètre `robots`), accumule les points reçus de tous les robots, puis effectue périodiquement la discrétisation et l'export de la carte, exactement comme le faisait l'ancien nœud unique, mais à partir de données provenant de plusieurs sources.

7.1.2 Gestion des topics et des repères TF entre robots

Faire fonctionner plusieurs robots simultanément a nécessité un travail de nommage spécifique, en plus de la séparation en deux nœuds décrite ci-dessus. D'une part, l'ensemble des topics propres à chaque robot (LiDAR, caméra, odométrie, mais aussi les topics de la chaîne de fusion présentés précédemment) a dû être placé dans un espace de noms (*namespace*) ROS2 dédié à ce robot, comme on le retrouve dans le code sous la forme du préfixe `/scout/...` ou `/ranger/...` devant chaque topic.

D'autre part, l'arbre de transformations TF a dû être repensé, car par défaut tous les robots publient leurs transformations sur les mêmes topics globaux `/tf` et `/tf_static`,

avec des noms de repères génériques qui entrent en conflit d'un robot à l'autre (par exemple `base_link` ou `odom` existant en double). Deux solutions ont été envisagées :

- séparer `/tf` et `/tf_static` en deux topics distincts par robot, ce qui revient à donner à chaque robot son propre arbre TF totalement indépendant. Cette solution est simple à mettre en place, mais interdit ensuite tout lien physique/spatial entre les arbres des différents robots, ce qui est problématique pour une cartographie coopérative où les robots doivent à terme pouvoir être situés les uns par rapport aux autres dans un référentiel commun ;
- préfixer directement les noms de repères eux-mêmes (par exemple `scout/base_link`, `ranger/base_link`) plutôt que de séparer les topics, en conservant un unique arbre TF partagé entre les robots.

C'est cette seconde solution qui a été retenue, car elle laisse la possibilité d'introduire plus tard un lien entre les repères des différents robots.L'ensemble des paquets et nœuds utilisés dans la chaîne doit être adapté pour tenir compte de ces nouveaux noms de repères préfixés plutôt que des noms par défaut.

7.1.3 Publication et agrégation des points étiquetés

Chaque `PerceptionNode` publie ses points fusionnés sur un topic qui lui est propre (`/<robot>/semantic_points`), sous forme de `PointCloud2` dont chaque point porte sa position, l'identifiant numérique de son label, une confiance liée à la portée de mesure et l'horodatage de la trame d'origine. La table de correspondance entre ces identifiants numériques et les noms de classes YOLO est publiée à part, une seule fois. L'identifiant numérique est transmis à la place du nom du label car le format `PointCloud2` n'accepte que des champs numériques pour chaque point, et non des chaînes de caractères.

Côté `MapFusionNode`, un jeu d'abonnements est créé dynamiquement pour chaque robot déclaré dans le paramètre `robots`. Les points reçus sont stockés séparément par robot, puis rassemblés en un seul jeu de données au moment de la génération de la carte. Cette agrégation permet à une même case de la grille (ou à un même voxel en 3D) de recevoir des points provenant de robots différents, et donc de produire une carte réellement commune.

7.1.4 Génération de carte périodique

Dans la version mono-robot, la génération de la carte n'était déclenchée qu'à l'arrêt du nœud. Dans `MapFusionNode`, cette étape est désormais exécutée périodiquement, à intervalle réglable (paramètre `write_interval_sec`), via un timer ROS2. Ce changement est nécessaire dès lors que le système fonctionne en temps réel et sur de longues durées avec plusieurs robots : une carte à jour doit être disponible en continu au cours de la mission, et non uniquement une fois celle-ci terminée.

Chapitre 8

Expérimentation et résultats

8.1 Expériences

8.1.1 Matériels expérimentaux

Afin de valider le framework développé sur des données réelles, en complément des données KITTI, deux robots terrestres de la gamme AgileX Robotics ont été utilisés pour réaliser des acquisitions sur le terrain : un Scout Mini et un Ranger.

Scout

Le Scout Mini [AgileX Robotics, 2026b] est une plateforme robotique terrestre compacte à quatre roues motrices, conçue pour la recherche et le développement. Chaque roue est entraînée indépendamment par un servomoteur, avec une suspension à double triangulation, pour une charge utile de 10 kg et une vitesse maximale de 1.5 m/s. Son autonomie est d'environ 3 heures, et elle peut franchir des pentes allant jusqu'à 30°. Le châssis est équipé de rails de fixation (T-slot) facilitant l'intégration de capteurs additionnels, et communique via un bus CAN.



FIGURE 8.1 – Robot Scout.

Ranger

Le Ranger [AgileX Robotics, 2026a] est une plateforme robotique terrestre à quatre roues indépendantes, offrant une motricité omnidirectionnelle. Elle dispose d'une charge utile bien supérieure à celle du Scout Mini, jusqu'à 150 kg, pour une vitesse maximale de 2.6 m/s et une autonomie d'environ 4 heures. Sa suspension à quatre roues indépendantes et son système de batteries modulaires interchangeables à chaud (*hot-swap*) permettent une utilisation prolongée sur des terrains accidentés.



FIGURE 8.2 – Robot Ranger.

Les deux robots sont équipés du kit de développement R&D proposé par Génération Robots pour la gamme AgileX. Ce kit regroupe l'ensemble des capteurs nécessaires à l'acquisition des données exploitées par le framework : un LiDAR, une caméra de profondeur

Intel RealSense D435, un récepteur GPS RTK, ainsi qu'une centrale inertielle (IMU). Il inclut également un calculateur embarqué et les pilotes ROS2 déjà installés et configurés pour l'ensemble de ces capteurs.

8.1.2 Configuration des acquisitions

Les deux robots n'ont pas été utilisés dans les mêmes conditions d'acquisition. Pour le Ranger, le bag exploité provient d'une acquisition multi-robots préexistante (un drone et le Ranger), réalisée par une autre personne et non par moi-même. Pour le Scout Mini, en revanche, j'ai pu piloter le robot avec l'aide de Gérald DHERBOMEZ (ingénieur de recherche et responsable technique de la plateforme PRETIL) et définir moi-même les conditions de l'acquisition en fonction des besoins du test : contrairement au bag du Ranger, d'une durée de 8 minutes, il fallait ici un bag plus court, facile à rejouer et à relancer, et surtout décrivant une véritable boucle fermée, afin de pouvoir tester la fermeture de boucle dans des conditions réelles.

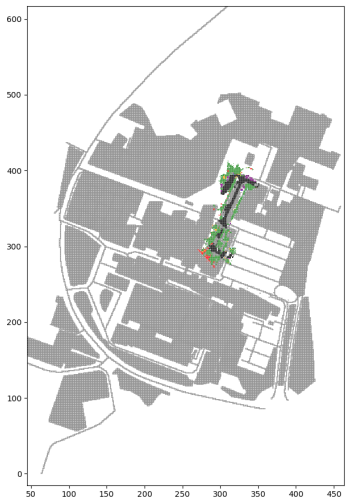
Les deux bags ont été acquis dans la même zone, à proximité du bâtiment ESPRIT, ce qui permet de disposer, pour les deux acquisitions, de la carte a priori correspondante.

Positions et orientations initiales

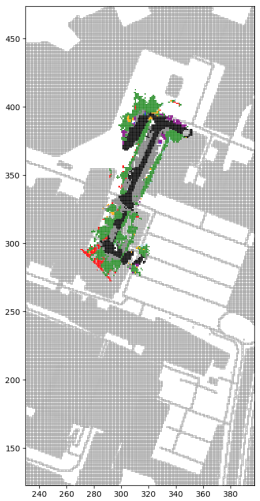
Pour placer la carte construite par le robot sur la carte a priori, il est nécessaire de connaître la position et l'orientation initiales du robot. Un GPS et un magnétomètre ont été utilisés à cette fin : le GPS présentait une erreur d'environ 5 mètres, et le magnétomètre s'est révélé inutilisable. L'orientation du robot a donc finalement été recalée en amont, visuellement. Un axe d'amélioration a été identifié à ce sujet, décrit dans le chapitre suivant, « Suites et améliorations ».

8.2 Résultats obtenus

8.2.1 Ranger

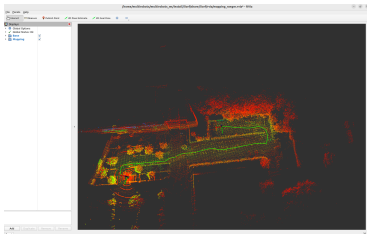


(a) Carte générée (pas de 1m).

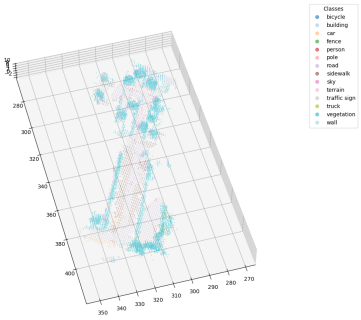


(b) Zoom sur la carte générée.

FIGURE 8.3 – Carte sémantique générée à partir du bag Ranger.



(a) Trajectoire produite par LIO.



(b) Carte générée en trois dimensions.

FIGURE 8.4 – Trajectoire LIO et carte 3D obtenues à partir du bag Ranger.

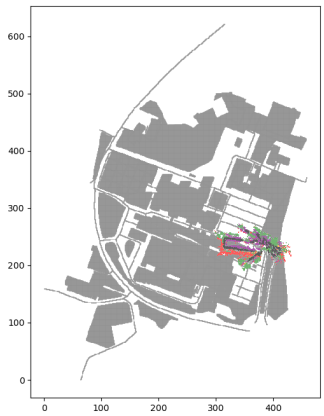
Sur les figures 8.3, on remarque que l'information sémantique associée aux cases est cohérente avec la réalité. On observe cependant un écart de position, dû à un manque de précision de la position initiale fournie par le GPS.

Sur la figure 8.4a, on peut comparer la trajectoire du robot avec la carte sémantique, et également identifier l'emplacement des bâtiments et des arbres, eux aussi représentés sur la carte sémantique.

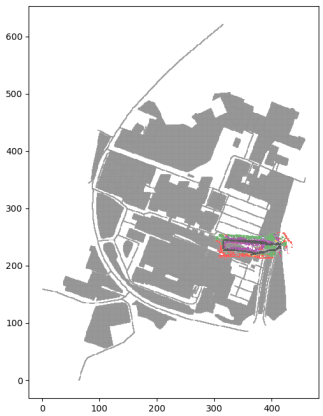
Sur la figure 8.4b, une carte en 3D a également été générée, où l'on visualise mieux les bâtiments et les arbres.

8.2.2 Scout

Cette acquisition du Scout, réalisée sur le parking devant le bâtiment ESPRIT, a permis d'effectuer un tour complet du parking à vitesse de marche, ce qui permet de tester la fermeture de boucle dans des conditions réelles.

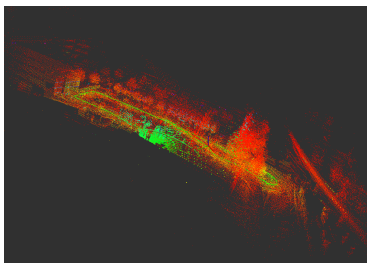


(a) Carte générée sans loop closure

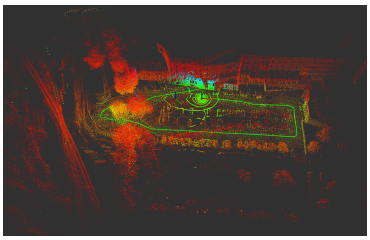


(b) Carte générée avec loop closure.

FIGURE 8.5 – Carte sémantique à partir du bag Scout, avec et sans fermeture de boucle.



(a) Trajectoire produite par LIO sans loop closure.



(b) Trajectoire produite par LIO avec loop closure.

FIGURE 8.6 – Trajectoire obtenues à partir du bag Scout, avec et sans fermeture de boucle.

Sur les figures présentant le test sans activation de la fermeture de boucle, on observe une séparation dans les données cartographiées. Celle-ci s'explique par la dérive de la trajectoire estimée par LIO.

On observe par ailleurs que cette dérive devient, à un moment de l'acquisition, particulièrement marquée : LIO parvient ensuite à retrouver une trajectoire visuellement cohérente et stable, mais avec une pente très exagérée, alors que le robot est resté à plat tout au long du trajet. Cette dérive importante pourrait s'expliquer par l'utilisation directe des données brutes de l'IMU, non filtrées ni lissées, contrairement à l'acquisition réalisée avec le Ranger, pour laquelle aucune dérive de ce type n'a été observée.

Avec la fermeture de boucle activée, on obtient une carte nettement plus cohérente avec le chemin réellement parcouru. Ceci s'explique par le fonctionnement même de la fermeture de boucle : dès qu'une contrainte de boucle est détectée (le robot revient à proximité d'un point déjà visité), l'optimiseur de graphe ne corrige pas seulement la dernière pose, mais redistribue la correction sur l'ensemble de la trajectoire passée. C'est précisément ce que fait le recalcul complet des points décrit précédemment.

On observe aussi que, sans fermeture de boucle, l'orientation en tout début de trajectoire correspond bien à l'orientation initiale renseignée manuellement. Avec la fermeture de boucle en revanche, on constate que même le début de la carte recalculée présente une orientation différente de celle renseignée manuellement. Cela s'explique par le fonctionnement de l'optimisation de graphe de LIO : lorsqu'une contrainte de boucle est ajoutée, l'optimiseur ne se contente pas de corriger les poses postérieures à sa détection, il recalcule l'erreur sur l'ensemble du graphe de poses, y compris les premières la pose de départ n'étant pas figée de manière rigide dans l'optimisation. C'est ce qui explique que l'orientation de la trajectoire corrigée, même à son origine, diffère légèrement de la valeur fixée manuellement.

8.2.3 Multi-robots

Le test multi-robots a été réalisé à partir des deux bags présentés précédemment (Ranger et Scout). Il ne s'agit donc pas d'un test multi-robots en temps réel, les deux acquisitions n'ayant pas été effectuées simultanément, mais il permet néanmoins de valider les bases du fonctionnement multi-robots : l'association des deux cartes générées, ainsi que l'absence de conflit entre les topics et les frames des deux robots, comme décrit dans le chapitre consacré au module multi-robots.

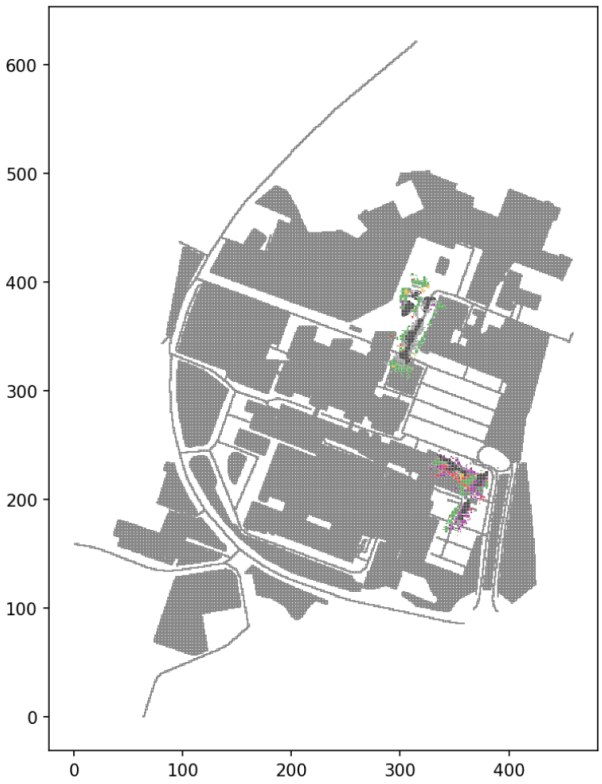


FIGURE 8.7 – Carte sémantique multirobots

On remarque, sur la figure 8.7, les deux cartes générées respectivement par le Ranger et le Scout Mini. Dans un premier temps, il est possible de valider l'agrégation de ces deux cartes en une carte commune, réunies dans un seul fichier GeoJSON.

Cependant, si une même case est visitée par les deux robots, les deux informations correspondantes sont conservées dans le GeoJSON sous la forme de deux lignes distinctes représentant la même case. L'une des suites possibles de ce projet, évoquée dans le chapitre suivant, consisterait à mettre en place une fusion bayésienne de ces informations redondantes.

Il est également possible d'activer la fermeture de boucle en configuration multi-robots. Cependant, dans la mesure où les deux bags ainsi que le framework pour les deux robots ont été exécutés simultanément sur la même machine, l'ajout de la fermeture de boucle entraîne une perte encore plus importante de données que ce que l'on peut constater déjà sans fermeture de boucle.

Chapitre 9

Suites et améliorations

9.1 Deux modèles modulaires

Un choix de conception essentiel adopté au cours de ce stage a été de traiter la brique de cartographie et la brique de détection et association sémantique comme deux modules indépendants. Cette approche modulaire permet de remplacer ou de faire évoluer l'une des deux briques sans impacter l'autre, et facilite les itérations de développement sur chacune séparément.

Ce choix présente cependant un inconvénient qui pourrait justifier une évolution architecturale si LIO venait à être confirmé comme le système de SLAM retenu pour la version finale. En l'état, le nuage de points LiDAR brut est traité indépendamment par le module de fusion sémantique, en parallèle du pipeline interne de LIO. Or LIO gère déjà nativement les fermetures de boucle et les corrections rétroactives de trajectoire sur ses propres structures de données internes ; les labels sémantiques, attribués en dehors de ce pipeline, n'en bénéficient donc pas automatiquement et doivent être resynchronisés séparément à chaque recalcul de carte.

Une évolution possible serait d'intégrer directement l'étiquetage sémantique au sein même du pipeline de LIO, en ajoutant le label comme une propriété supplémentaire portée nativement par chaque point (au même titre que ses coordonnées), plutôt que de le gérer dans un module externe. Les points conserveraient ainsi leurs propriétés sémantiques directement à travers les structures internes de LIO, et bénéficieraient automatiquement des corrections de trajectoire lors des fermetures de boucle, sans mécanisme de resynchronisation additionnel.

9.2 Position et orientation initiale

Comme expliqué précédemment, lors de l'initialisation du robot dans la carte a priori, un décalage spatial a été constaté entre la position donnée par le GPS et la position réelle du robot. Ce décalage est dû à l'imprécision du GPS, mais pourrait être amélioré en attendant plusieurs mesures GPS successives et en calculant leur moyenne plutôt qu'en se fiant à une seule mesure instantanée.

Concernant l'orientation initiale, le magnétomètre a été utilisé, mais les valeurs obtenues se sont révélées complètement erronées. Une piste d'amélioration serait de créer un script simple, à lancer avant le déploiement du robot, qui ferait avancer ce dernier de quelques mètres afin de déterminer sa position et son orientation initiales avec le GPS, à partir de ce déplacement plutôt que du magnétomètre.

L' évolution en un modèle (expliqué précédemment) permettrait aussi de pouvoir facilement modifier les positions et orientations initiales des robots au préalable ce qui nous permetrait de ne plus transformer les points pour le décalage mais le robot directement et donc d'avoir un algorithme mieux optimisé.

9.3 Fusion Bayésienne

Une des améliorations envisagées pour la suite de ce travail concerne la façon dont les cartes sont combinées entre elles. Actuellement, l'agrégation des points que ce soit entre plusieurs robots ou entre la carte construite par fusion LiDAR-caméra et la carte a priori se fait par simple agrégation. L'objectif initial était de remplacer cette agrégation simple par une fusion bayésienne, qui permettrait de combiner ces différentes sources d'information en tenant compte de leur fiabilité respective plutôt que de ne garder que le label majoritaire. Faute de temps, cette piste n'a pas pu être mise en œuvre au cours de ce stage et reste une perspective pour la suite du projet.

9.4 Tests en conditions réelles

Pour aller vers un test en conditions réelles, plusieurs étapes restent à mener. La première consisterait à tester chaque robot indépendamment en temps réel, ce qui n'a pas été fait durant ce stage : seules des acquisitions ont été réalisées, permettant ensuite de rejouer les données enregistrées pour les tests. Un test en temps réel sur chaque robot serait un bon moyen de vérifier que le framework peut tourner directement sur le calculateur embarqué du robot. Une fois cette étape validée pour chaque robot séparément, elle pourrait être étendue au fonctionnement multi-robots.

9.4.1 Entraînement de YOLO

Pour tester le framework dans un environnement différent, comme une forêt, il faudrait également ré-entraîner le modèle YOLO avec un jeu de données adapté à ce type d'environnement. Actuellement, le modèle YOLO utilisé est entraîné sur Cityscapes, un jeu de données composé principalement de scènes urbaines. Dans un environnement forestier, la labellisation sémantique serait donc probablement peu fiable, le modèle n'ayant jamais été exposé à ce type de classes.

Chapitre 10

Conclusion

Ce stage avait pour objectif de développer une brique de cartographie sémantique coopérative pour des flottes de robots mobiles évoluant en environnement extérieur, dans le cadre du projet ANR-SOS. Le framework développé permet de projeter les nuages de points LiDAR sur les images caméra, de les étiqueter sémantiquement à l'aide de YOLO, puis de les accumuler dans le repère monde grâce à l'odométrie LiDAR-inertielle fournie par LIO, avant de générer une carte sémantique exportée au format GeoJSON.

Sur le plan expérimental, l'approche a d'abord été validée sur le jeu de données KITTI, puis sur des acquisitions réelles réalisées avec deux robots terrestres AgileX, le Scout Mini et le Ranger.

Ce stage constitue ainsi une première base fonctionnelle et validée pour la cartographie sémantique multi-robots, tout en laissant ouvertes plusieurs pistes de travail identifiées et détaillées dans le chapitre précédent, en particulier la fusion bayésienne des informations redondantes, l'automatisation du recalage initial du robot, et la validation en temps réel sur les plateformes embarquées. Ce travail s'inscrit enfin dans une logique de continuité pour le projet ANR-SOS, en proposant une base capable d'accueillir les futures contributions du laboratoire.

Chapitre 11

Annexe


```
{
  "type": "FeatureCollection",
  "name": "export_grid",
  "crs": { "type": "name", "properties":
    { "name": "urn:ogc:def:crs:EPSG::2154" } },
  "features": [
    { "type": "Feature",
      "properties": { "center_x": 0.5, "center_y": 159.5,
        "label": "road", "id": "way/154556229" },
      "geometry": { "type": "Polygon", "coordinates": [ [
        [ 0.0, 159.0 ], [ 1.0, 159.0 ],
        [ 1.0, 160.0 ], [ 0.0, 160.0 ],
        [ 0.0, 159.0 ] ] ] } },
    { "type": "Feature",
      "properties": { "center_x": 1.5, "center_y": 159.5,
        "label": "road", "id": "way/154556229" },
      "geometry": { "type": "Polygon", "coordinates": [ [
        [ 1.0, 159.0 ], [ 2.0, 159.0 ],
        [ 2.0, 160.0 ], [ 1.0, 160.0 ],
        [ 1.0, 159.0 ] ] ] } },
    ...
  ]
}
```

Bibliographie

AgileX Robotics. Ranger – omnidirectional unmanned ground vehicle. https://global.agilex.ai/products/ranger, 2026a. Consulté en 2026.

AgileX Robotics. Scout mini – compact unmanned ground vehicle. https://global.agilex.ai/products/scout-mini, 2026b. Consulté en 2026.

Chunge Bai, Tao Xiao, Yajie Chen, Haoqian Wang, Fang Zhang, and Xiang Gao. Faster-LIO : Lightweight tightly coupled lidar-inertial odometry using parallel sparse incremental voxels. *IEEE Robotics and Automation Letters*, 7(2) :4861–4868, 2022. doi : 10.1109/LRA.2022.3152830.

Carlos Campos, Richard Elvira, Juan J. Gómez Rodríguez, José M. M. Montiel, and Juan D. Tardós. ORB-SLAM3 : An accurate open-source library for visual, visual-inertial, and multimap SLAM. *IEEE Transactions on Robotics*, 37(6) :1874–1890, 2021. doi : 10.1109/TRO.2021.3075644.

Kenny Chen, Ryan Nemiroff, and Brett T. Lopez. Direct LiDAR-inertial odometry : Lightweight LIO with continuous-time motion correction. In *2023 IEEE International Conference on Robotics and Automation (ICRA)*, 2023.

Xieyuanli Chen, Andres Milioto, Emanuele Palazzolo, Philippe Giguère, Jens Behley, and Cyrill Stachniss. SuMa++ : Efficient LiDAR-based semantic SLAM. In *2019 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2019.

Frank Dellaert. Square root SAM. In *Robotics : Science and Systems (RSS)*, 2005.

Martin Fleischmann, Joris Van den Bossche, Kelsey Jordahl, Matthew John Richards, James McBride, Jacob Wasserman, Brendan Ward, and Levi John Wolf. GeoPandas : Fundamental data structures for vector spatial data in Python. *Computers, Environment and Urban Systems*, 130 :102495, 2026. doi : 10.1016/j.compenvurbsys.2026.102495.

Andreas Geiger, Philip Lenz, Christoph Stiller, and Raquel Urtasun. Vision meets robotics : The KITTI dataset. *The International Journal of Robotics Research*, 32(11) : 1231–1237, 2013.

Dongjiao He, Wei Xu, Nan Chen, Fanze Kong, Zheng Yuan, and Fu Zhang. Point-LIO : Robust high-bandwidth lidar-inertial odometry. *Advanced Intelligent Systems*, 2022. doi : 10.1002/aisy.202200459.

Wolfgang Hess, Damon Kohler, Holger Rapp, and Daniel Andor. Real-time loop closure in 2D LIDAR SLAM. In *2016 IEEE International Conference on Robotics and Automation (ICRA)*, pages 1271–1278, 2016. doi : 10.1109/ICRA.2016.7487258.

Glenn Jocher, Jing Qiu, Mengyu Liu, Shuai Lyu, Fatih Cagatay Akyon, and Muhammet Esat Kalfaoglu. Ultralytics yolo26 : Unified real-time end-to-end vision models, 2026. URL https://arxiv.org/abs/2606.03748.

Mathieu Labbé and François Michaud. RTAB-Map as an open-source lidar and visual simultaneous localization and mapping library for large-scale and long-term online operation. *Journal of Field Robotics*, 36(2) :416–446, 2019. doi : 10.1002/rob.21831.

OpenStreetMap contributors. Planet dump. https://www.openstreetmap.org, 2026.

Tong Qin, Peiliang Li, and Shaojie Shen. VINS-Mono : A robust and versatile monocular visual-inertial state estimator. *IEEE Transactions on Robotics*, 34(4) :1004–1020, 2018. doi : 10.1109/TRO.2018.2853729.

Martin Raifer. overpass turbo. https://overpass-turbo.eu/, 2013.

Antoni Rosinol, Marcus Abate, Yun Chang, and Luca Carlone. Kimera : an open-source library for real-time metric-semantic localization and mapping. In *2020 IEEE International Conference on Robotics and Automation (ICRA)*, 2020.

Tixiao Shan and Brendan Englot. LeGO-LOAM : Lightweight and ground-optimized lidar odometry and mapping on variable terrain. In *2018 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pages 4758–4765, 2018. doi : 10.1109/IROS.2018.8594299.

Tixiao Shan, Brendan Englot, Drew Meyers, Wei Wang, Carlo Ratti, and Daniela Rus. Lio-sam : Tightly-coupled lidar inertial odometry via smoothing and mapping. In *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pages 5135–5142. IEEE, 2020. URL https://arxiv.org/abs/2007.00258.

Pauli Virtanen, Ralf Gommers, Travis E. Oliphant, Matt Haberland, Tyler Reddy, David Cournapeau, Evgeni Burovski, Pearu Peterson, Warren Weckesser, Jonathan Bright, Stéfan J. van der Walt, Matthew Brett, Joshua Wilson, K. Jarrod Millman, Nikolay Mayorov, Andrew R. J. Nelson, Eric Jones, Robert Kern, Eric Larson, C. J. Carey, İlhan Polat, Yu Feng, Eric W. Moore, Jake VanderPlas, Denis Laxalde, Josef Perktold, Robert Cimrman, Ian Henriksen, E. A. Quintero, Charles R. Harris, Anne M. Archibald, Antônio H. Ribeiro, Fabian Pedregosa, Paul van Mulbregt, and SciPy 1.0 Contributors. SciPy 1.0 : Fundamental algorithms for scientific computing in Python. *Nature Methods*, 17 :261–272, 2020. doi : 10.1038/s41592-019-0686-2.

Ignacio Vizzo, Tiziano Guadagnino, Benedikt Mersch, Louis Wiesmann, Jens Behley, and Cyrill Stachniss. KISS-ICP : In defense of point-to-point ICP – simple, accurate, and robust registration if done the right way. *IEEE Robotics and Automation Letters*, 2023. doi : 10.1109/LRA.2023.3236571.

Wei Xu, Yixi Cai, Dongjiao He, Jiarong Lin, and Fu Zhang. FAST-LIO2 : Fast direct LiDAR-inertial odometry. *IEEE Transactions on Robotics*, 38(4) :2053–2073, 2022. doi : 10.1109/TRO.2022.3141876.

YJZLuckyBoy. liorf : modified version of lio-sam with the feature extraction module removed. https://github.com/YJZLuckyBoy/liorf, 2022.

Ji Zhang and Sanjiv Singh. LOAM : Lidar odometry and mapping in real-time. In *Robotics : Science and Systems X*, 2014. doi : 10.15607/RSS.2014.X.007.